

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Algorithms for small, noisy, near-future quantum computers

(Thesis Proposal)

Mgr. Denisa Lampášová

Supervisor: Mgr. Daniel Nagaj, Ph.D.

Bratislava, 2022

Abstract

The current development of quantum computers shows there is still a long way towards fully scalable, universal, fault-tolerant systems. On the other hand, current technological possibilities show that even relatively small systems with noisy control are reaching the boundaries of current classical simulability. In the proposed work, we want to understand the potential and limits of such systems for practical purposes. We want to show that they are not only classically hard to simulate, but that they are also useful. We will investigate the limits and potential for state preparation, constraint optimization, quantum simulations, as well as machine learning applications.

Keywords: local Hamiltonians, ground state connectivity, stabilizer codes, local constraint optimization, approximate solutions, quantum algorithms, quantum supremacy

Contents

Notational Conventions **iii**

Introduction **1**

1 A geometric approach to quantum circuit complexity **5**
1.1 The Hilbert space is enormous 5
1.2 GSCON: Ground state connectivity 12

2 Algorithms for NISQ devices **19**
2.1 QML: Quantum Machine Learning 19
2.2 QAOA: The Quantum Approximate Optimization Algorithm 21
2.2.1 Combinatorial Optimization Problems (COPs) 21
2.2.2 QAOA 24
2.2.3 Local classical algorithms 31

3 Boson Sampling **32**
3.1 Bosonic second quantization formalism 32
3.2 `BosonSampling` 34

4 Perspectives **43**

Bibliography **46**

Appendix A Time complexity, a brief overview **54**
A.1 Big $\mathcal{O}$ notation and time complexity 54
A.2 Complexity classes 60

Notational Conventions

- We use the convention, where $\hbar = 1$.
- The Pauli matrices/operators:

$$\mathbb{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \text{NOT},$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = iXZ.$$

- The computational basis (the Z (eigen)basis):

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \& \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

- The X basis:

$$|+\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + |1\rangle\right) = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix},$$
$$|-\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle - |1\rangle\right) = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

- In the case of 2 or more qubits, we often leave the tensor product symbol $\otimes$ out and abbreviate, e.g. $|0\rangle \otimes |1\rangle \otimes |0\rangle \otimes |1\rangle = |0\rangle\,|1\rangle\,|0\rangle\,|1\rangle = |0101\rangle$.
- When writing a tensor product of single-qubit operators, we usually erase all identities and tensor product symbols. For the remaining single-qubit operators, we specify their location in the tensor product (= which qubit the single-qubit operator acts on) by a subscript. For example:

$$X \otimes \mathbb{I} = X_1,$$
$$\mathbb{I} \otimes Z \otimes \mathbb{I} \otimes \mathbb{I} \otimes Z = Z_2 Z_5.$$

- For the identity matrix $\mathbb{I}_d$, we usually do not specify the dimension d as it is clear from the context. Hence, even though we defined $\mathbb{I}$ as a 2-dimensional identity, we write $\mathbb{I}$ for any 2^n-dimensional identity $\underbrace{\mathbb{I} \otimes \mathbb{I} \otimes \cdots \otimes \mathbb{I}}_{n} = \mathbb{I}$.

Introduction

"Perhaps, for better understanding of this phenomenon [DNA replication], we need a mathematical theory of quantum automata. Such a theory would provide us with mathematical models of deterministic processes with quite unusual properties. One reason for this is that the quantum state space has far greater capacity than the classical one: for a classical system with N states, its quantum version allowing superposition accommodates c^N states. When we join two classical systems, their number of states N_1 and N_2 are multiplied, and in the quantum case we get exponential growth $c^{N_1 N_2}$. These crude estimates show that the quantum behavior of the system might be much more complex than its classical simulation."

— Yuri Manin, *Computable and Uncomputable (1980)*

"I'm not happy with all the analyses that go with just the classical theory, because nature isn't classical, dammit, and if you want to make a simulation of nature, you'd better make it quantum mechanical, and by golly it's a wonderful problem, because it doesn't look so easy."

— Richard Feynman, *Simulating Physics with Computers (1982)*

"The picture that'd emerged over the last quarter century is more subtle than any science-fiction writer would have had the imagination to make up. If you were just making up a science-fiction supercomputer, you would just say: 'Oh, it can solve NP-complete problems, it can solve the halting problem, whatever...' You wouldn't make up something that can do factoring but not finding short vectors in lattices."

— Scott Aaronson, *https://youtu.be/OysWaeMt8Io?t=465 (2018)*

Since Paul Benioff proposed a quantum mechanical model of the Turing machine in 1980 [8], and Yuri Manin [53], as well as Richard Feynman [29], suggested that quantum computers have the potential to simulate things a classical computer could not feasibly do, many milestones toward fault-tolerant quantum computing were reached:

1992 The Deutsch–Jozsa algorithm [18] illustrates that an exponential-time speedup might be possible.

1994 Peter Shor introduces a polynomial-time algorithm for discrete logarithm and factoring [66].[1] This attracts attention as important public-key cryptography schemes, such as RSA, The Finite Field Diffie-Hellman, and The Elliptic Curve Diffie-Hellman key exchange, based their security on the assumption that no efficient algorithm to break them exists. However, quantum computers large enough to perform meaningful instances of these problems are likely many years away.

1995 Shor proposes the first schemes for quantum error correction [67].

1996 Lov Grover, at Bell Labs, invents a search algorithm [35] that allows us to find an item in an (unstructured) database in $\mathcal{O}(\sqrt{n})$ steps as opposed to classical algorithms that cannot guarantee better performance than $\mathcal{O}(n)$. Quantum computing might thus provide a quadratic speedup for a wide variety of problems.

1998 Isaac Chuang, Neil Gershenfeld, and Mark Kub created the first two-qubit quantum computer that could perform computations [16].

2011 The first commercially available quantum computer system (running on a 128-qubit processor designed to implement a special-purpose quantum annealing [46, 30, 62, 17]) is offered by D-Wave Systems.

2019 In partnership with the U.S. National Aeronautics and Space Administration (NASA), Google AI claims to achieve quantum supremacy [3]. Their 53-qubit Sycamore chip performed in around 3 minutes a quantum computation that would take roughly 2.5 days to simulate on the world's best supercomputer at the time [60]. Whether they indeed achieved quantum supremacy - performed a computational task that no classical computer can efficiently simulate, and thus refuted the extended Church-Turing thesis - is a topic of active research.[2]

Today, IBM provides cloud access to quantum computers with up to 127 qubits. Rigetti's Aspen processors go up to 40 qubits. IONQ introduced a 32-qubit quantum computer which they expect to provide 22 Algorithmic Qubits (AQ). D-Wave's Pegasus is now at 5600 special-purpose qubits, discovering the potential of quantum-annealing for, e.g., optimization or drug discovery. And here we have named only a few companies. Not to mention an increasing number of quantum software start-ups (1QBit, Zapata Computing, Phasecracf, etc.).

[1] To this day, it is unknown whether an efficient classical algorithm to solve either of them exists.

[2] Since then, more advanced quantum supremacy experiments have been accomplished [78, 77, 74, 79].

We are currently approaching, as John Preskill coined it [61], the *Noisy intermediate-scale quantum (NISQ)* era. Era that is *noisy*, as quantum processors are not yet advanced enough to reach fault-tolerance, and *intermediate-scale*, as the leading quantum processors contain about 50 to a few hundred qubits and thus are not large enough to profit sustainably from quantum supremacy. But there are promising algorithms, such as the *Variational quantum eigensolver (VQE)* or *the Quantum approximate optimization algorithm (QAOA)*, considered to be suitable for NISQ devices.

We have decided to start this thesis proposal by reviewing some technical aspects of quantum computation. It is a challenging task to ensure quantum properties of any system, being frail as they are, and to avoid quantum decoherence. Therefore, quantum hardware is inevitably designed to be highly system-specific, and thus, manipulation of the quantum system's state is restricted only to a carefully chosen universal set of unitary transformations. Even so, the errors increase exponentially with the circuit depth. In the first chapter, we'll shortly discuss how many states can be reached within given circuit depth, whether we can find optimal circuits knowing the states we want to reach, and how to compile a general unitary to a chosen gate set. We will also look at the ground state connectivity problem in the following context. One needs to reliably store quantum information and ensure some level of reliability of a computation. Therefore, it is helpful to know how frail the quantum states are, for example, whether they can change their logical value without being detected.

In chapter 2, we will discuss the potential applications of NISQ devices. We shortly review the current state of the matter: *What suitable algorithms are there? To which problems are they applied, and how do they hold up against the classical competition?* Most of the chapter will be devoted to QAOA, which is the main subject of our proposed study. We are interested in its performance guarantees, in particular, compared to local classical algorithms, which, so far, seem to match/outrun it [38, 39].

Chapter 3 is devoted to BosonSampling, a linear-optical task that is #P-hard to simulate but experimentally accessible and thus a reasonable and successful [78, 77] candidate for demonstrating quantum supremacy. We will briefly introduce bosonic second quantization formalism, define BosonSampling and provide a detailed analysis of several simple BosonSampling instances.

We state in detail and analyze the following questions (the subject of the author's work in her continuing doctoral studies) in Chapter 4.

- What properties does QAOA have that make it worthwhile to be used for finding approximate solutions to combinatorial optimization problems?
- Are there problems where QAOA offers better performance guarantees than local classical algorithms proposed by Hastings [38]?

- Given a stabilizer-code Hamiltonian and two ground states of this Hamiltonian, can we efficiently decide whether these two states can be connected via the ground space in a polynomial number of steps, or is there inevitably some energy barrier? Is the *ground state connectivity problem* [32] restricted to stabilizer-code Hamiltonians (or some other simpler case) in the complexity class P?

- Is there a quantity that imposes a partial order on BosonSampling instances, which sheds light on their relative complexity?

If any unclarities in notation should arise while reading this thesis proposal, we recommend taking a look at the Notational Conventions section. Moreover, in the appendix Computational complexity, a brief overview, we explain the big $\mathcal{O}$ notation and describe the complexity classes that appear throughout the text.

1 A geometric approach to quantum circuit complexity

"I think one of the reasons that people have not had powerful ways to think about complexity and in particular for connecting it with physics has been an insufficient appreciation of its geometric character."

— Leonard Susskind , https://youtu.be/6OXdhV5BOcY?t=1311

1.1 The Hilbert space is enormous

In this section, we wish to study the geometry and complexity of quantum states. In particular, we want to show that exponentially many quantum states require exponential circuit depth to be synthesized.

A general (pure) state in an n-qubit Hilbert space can be expressed as

$$|\psi\rangle = \sum_{i=0}^{N-1} \alpha_i |i\rangle ,$$

where $N = 2^n$ is the dimension of the Hilbert space and α_i, $i \in \{0, 1, \ldots, N-1\}$, are complex coefficients satisfying the normalization condition: $\sum_{i=0}^{N-1} |\alpha_i|^2 = 1$. From the geometry and complexity point of view, we can state two facts. First, the state space is the sphere S^{2N-1} embedded in $\mathbb{C}^N$. Second, to specify a single n-qubit state, one needs exponentially many, $2N-1$, real numbers.

Now, let us ask: *How many states are there altogether?* Of course, as there is a continuous infinity of the coefficients, there is a continuous infinity of states. We can get more insight by regularizing the space. Let us assume the amplitudes are chosen from a discrete set of m values: $\alpha_i \in \{v_1, v_2, \ldots, v_m\}$. For simplicity, we do not require the states to be normalized.[1] Then, there would be m^{2^n} possible states. To grasp this number better, let us substitute in some small values:

[1] We can normalize them afterward if needed.

$\#\mathrm{s} = m^{2^n} = ?$	$m = 3$	$m = 4$	$m = 5$
$n = 3$	$3^{2^3} \approx 6.6 \cdot 10^3$	$4^{2^3} \approx 6.6 \cdot 10^4$	$5^{2^3} \approx 3.9 \cdot 10^5$
$n = 4$	$3^{2^4} \approx 4.3 \cdot 10^7$	$4^{2^4} \approx 4.3 \cdot 10^9$	$5^{2^4} \approx 1.5 \cdot 10^{11}$
$n = 5$	$3^{2^5} \approx 1.9 \cdot 10^{15}$	$4^{2^5} \approx 1.8 \cdot 10^{19}$	$5^{2^5} \approx 2.3 \cdot 10^{22}$

Notice that the logarithm of the number of states grows rapidly with the number of qubits n and very weakly with the regularizer m:

$$\log \#\mathrm{s} = \log m^{2^n} = 2^n \log m.$$

We can do a more precise counting. As mentioned above, the state space of an n-qubit system is a $(2N-1)$-dimensional topological sphere S^{2N-1}. But if the system is isolated, the overall $U(1)$ phase is not measurable, and therefore, there is some redundancy. Achieving no redundancy, the space of normalized n-qubit states is the complex projective space of (complex) dimension $N-1$: $CP(N-1) = S^{2N-1}/S^1$. Now, let us introduce the following regularization. We coarse-grain the space into balls with radius ε (ε-balls). Any state is identified with an ε-ball it's located in. As well, in reality, it's almost impossible to distinguish two states that are too close in the Hilbert norm.

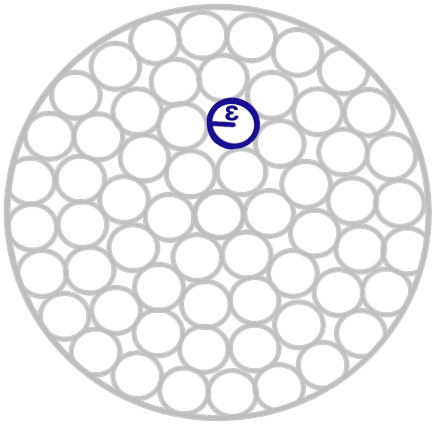


Figure 1.1: $CP(N-1)$ divided into cells of linear size ε.

This time, the question is: *How many ε-balls are there in $CP(N-1)$?* We need to divide the volume of $CP(N-1)$,

$$\mathrm{vol}\Big(CP(N-1)\Big) = \frac{\pi^{N-1}}{(N-1)!},$$

by the volume of an $2(N-1)$-dimensional ball of radius ε,

$$\mathrm{vol}\Big(\varepsilon\text{-ball}\Big) = \frac{\pi^{N-1}}{(N-1)!} \cdot \varepsilon^{2N-2}.$$

So, there are

$$\frac{\mathrm{vol}\Big(CP(N-1)\Big)}{\mathrm{vol}\Big(\varepsilon\text{-ball}\Big)} = \frac{1}{\varepsilon^{2N-2}}$$

states in the $CP(N-1)$. Observe that the logarithm of the number of states,

$$\log \#\mathrm{s} \approx 2^n \log \frac{1}{\varepsilon}$$

follows the same pattern as before. It grows very rapidly with the number of qubits n, whereas very weakly with the regularizer $\frac{1}{\varepsilon}$.

Now, let us think for a moment about the size of the Hilbert space from a different point of view: *How far apart can two states* $|A\rangle$ *and* $|B\rangle$ *be?* To start, we should define what we mean by *the distance between two states.*

The usual Hilbert-Schmidt inner product metric has a significant drawback. Consider the following two states: $|A\rangle = |\psi\rangle \otimes |0\rangle$ and $|B\rangle = |\psi\rangle \otimes |1\rangle$, where $|\psi\rangle$ might be a complicated very-large-number-of-qubit state. These two states are orthogonal, and thus, in the inner product, as far apart as possible. However, in some sense, they are not very different; they only differ by the orientation of a single qubit. From the geometric point of view, a more granulated separation would be more useful. Therefore, here, we'll use the *relative complexity* metric.

Let us define relative complexity $\mathcal{C}$. For any two states $|A\rangle$ and $|B\rangle$, there exists a unitary U such that $|A\rangle = U\,|B\rangle$. Let $\mathcal{G}$ be a universal set of quantum gates. We define $m_{\mathcal{G}}(U)$ to be the minimal number of gates from $\mathcal{G}$ required to synthesize U. Relative complexity $\mathcal{C}_{\mathcal{G}}\left(|A\rangle, |B\rangle\right)$ is equal to $m_{\mathcal{G}}(U)$.

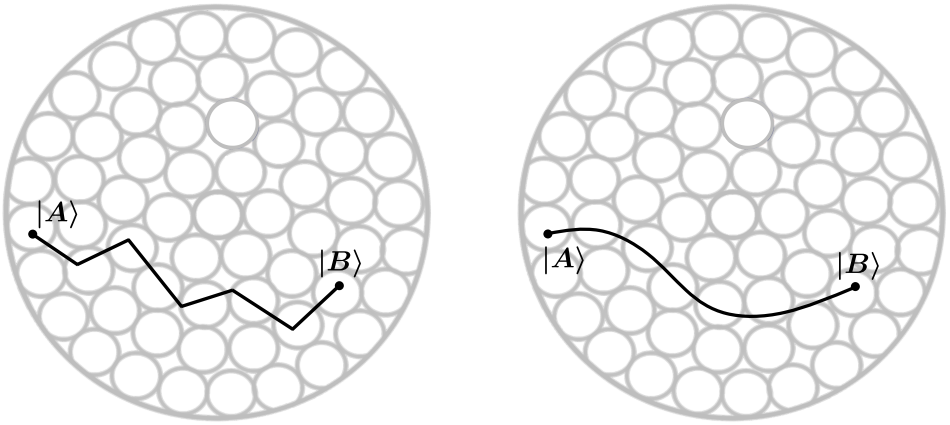


Figure 1.2: We identify relative complexity with the shortest path between the two states, whether we connect them via discrete quantum circuit (the case on the left) or Hamiltonian evolution (the case on the right).

What properties does relative complexity have?

- $\mathcal{C} \geq 0$. The equality $\mathcal{C} = 0$ holds if and only if $|A\rangle = |B\rangle$.

- It satisfies the triangle inequality: $\mathcal{C}_{\mathcal{G}}\left(|A\rangle, |B\rangle\right) \leq \mathcal{C}_{\mathcal{G}}\left(|A\rangle, |C\rangle\right) + \mathcal{C}_{\mathcal{G}}\left(|C\rangle, |B\rangle\right)$. Because one can always get from $|A\rangle$ to $|B\rangle$ through an intermediate state $|C\rangle$. But unless the optimal path leads through $|C\rangle$, there exists a shorter path.

- We will demand $\mathcal{G}$ to be inverse-closed. That is, whenever g is an allowed gate, so is $g^\dagger$. This will ensure $m_{\mathcal{G}}(U) = m_{\mathcal{G}}(U^\dagger)$, and for that reason, relative complexity will be a symmetric function:

$$\mathcal{C}_{\mathcal{G}}\left(|A\rangle, |B\rangle\right) = \mathcal{C}_{\mathcal{G}}\left(|B\rangle, |A\rangle\right).$$

Therefore, relative complexity is indeed a metric.

We can analogically define relative complexity for a pair of unitary operators:

$$\mathcal{C}_{\mathcal{G}}\left(U, V\right) \equiv m_{\mathcal{G}}(UV^\dagger) = m_{\mathcal{G}}(VU^\dagger) \equiv \mathcal{C}_{\mathcal{G}}\left(V, U\right),$$

as $U = (UV^\dagger)V$. In this case, we can easily observe that relative complexity is a right-invariant metric:

$$\forall W \in \mathcal{U}(2^n): \quad \mathcal{C}(U, V) = \mathcal{C}(UW, VW).$$

There exists a unitary G, such that $U = GV$. The unitaries UW and VW are connected via the same unitary: $UW = GVW$. It follows that $\mathcal{C}$ is right-invariant.

So far, the main point is that the distance between states is based on circuit complexity; it is the minimal number of gates from a universal set of gates $\mathcal{G}$ required to transform one of these states to the other one. Let us find out how many states can we reach within a single step. *How many different kinds of depth-1 circuits are there?*

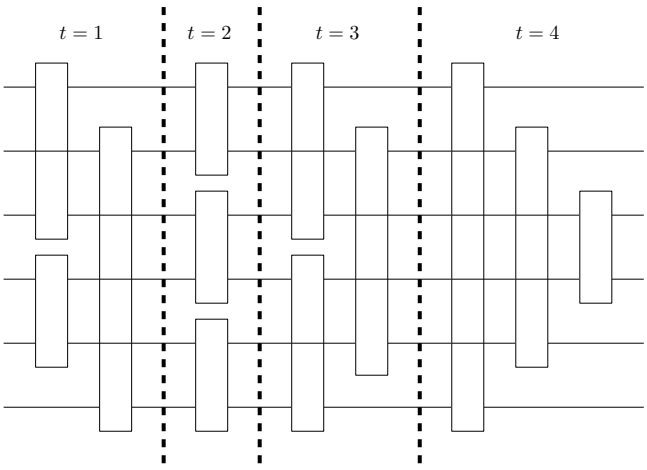


Figure 1.3: An example 6-qubit depth-4 circuit. In each step, each qubit is involved at most (in our case exactly) in one gate.

For simplicity, let us assume that:

- $\mathcal{G}$ consists of k purely 2-local gates,
- each $g \in \mathcal{G}$ depends on the order of the qubits (we will be dividing the qubits into ordered pairs),
- each qubit is involved in some gate in each step. It implies that, in each step, exactly $\frac{n}{2}$ gates is simultaneously applied.

The final form of a depth-1 circuit depends on how the qubits were paired, and what gate was used for each pair.

Let us start with counting how many different ordered pairings of all the qubits are there. We first determine the order of the qubits, i.e., assign each qubit whether it will be the first one in the pair, or the second one. There is $\binom{n}{\frac{n}{2}}$ ways to do it. Now, we can imagine the qubits which were assigned to be the second ones (the "second" qubits) to represent slots. There is as many different ways to match the "first" qubits with the "second" qubits as there is different ways to place the $\frac{n}{2}$ "first" qubits into the $\frac{n}{2}$ slots. Namely, $\frac{n}{2}!$ ways. So, in total, there is

$$\binom{n}{\frac{n}{2}} \cdot \frac{n}{2}! = \frac{n!}{\frac{n}{2}!}$$

ways to pair up all the qubits for each step. Using Stirling's approximation:

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n,$$

we can simplify the result:

$$\frac{n!}{\frac{n}{2}!} \approx \sqrt{2}\left(\frac{2}{e}\right)^{\frac{n}{2}} \cdot n^{\frac{n}{2}} \approx n^{\frac{n}{2}}.$$

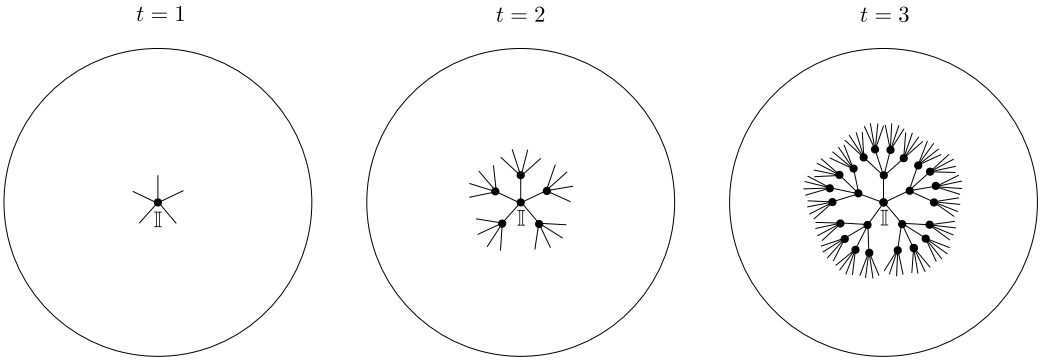
Once the qubits were divided, there is k distinct gates that can be applied to each of the $\frac{n}{2}$ qubit pairs, and thus, $k^{\frac{n}{2}}$ different depth-1 circuits for each fixed pairing. At last, the total number of different kinds of depth-1 circuits is

$$\overbrace{n^{\frac{n}{2}}}^{\text{the number of different pairings}} \cdot \underbrace{k^{\frac{n}{2}}}_{\text{the number of depth-1 circuits for each pairing}}.$$

In other words, there are $d = (nk)^{\frac{n}{2}}$ different unitary operators we can move with. We denoted this quantity by d to indicate it will represent the degree of a graph, i.e., the number of edges that are incident to each vertex of the graph.

How many states can we reach in m steps? The depth-0 circuit is identity. In one step, we can move with d different unitary operators, and thus, reach d new states. From each new state we reached, we can again move with any depth-1 circuit. One of the circuits will be the inverse to the previous one, but the remaining $d-1$ circuits

will introduce new states. Hence, the second step adds $d \cdot (d-1)$ states. All in all, in two steps, we can reach $d + d \cdot (d-1)$ states. As the total number of states is large, up until we reach all the states, the probability of returning to a state we've already visited is small. Therefore, all the time, we expect no collisions. We can depict this progress as a d-regular tree with the root being the state we start in. The depth a state $|\alpha\rangle$ is located at in this tree is the relative complexity between $|\alpha\rangle$ and the root state. Alternatively, the root can be the identity matrix and the vertices the available circuits, i.e., the elements of $SU(2^n)$:



Then, the depth a circuit U is located in is relative complexity between U and the identity $\mathcal{C}(\mathbb{I}, U)$. Let us continue counting. The third step provides extra $d \cdot (d-1)^2$ circuits, leaving us with $d + d \cdot (d-1) + d \cdot (d-1)^2$ available states we can reach in three steps. With m steps, we have the reach of

$$d \cdot \sum_{j=0}^{m-1} (d-1)^j \quad \overset{\text{sum of geometric sequence}}{=} \quad d \cdot \frac{(d-1)^m - 1}{d-1} \approx d^m = (nk)^{nm/2}$$

states. We presume d is very large.

Recall that relative complexity $\mathcal{C}(\mathbb{I}, U)$ is the minimal number of gates required to synthesize U, and that each step consists of exactly $\frac{n}{2}$ gates. Therefore, $\frac{nm}{2}$ is relative complexity $\mathcal{C}$. The number of all reachable states within m steps is

$$(nk)^{\mathcal{C}} = e^{\mathcal{C} \log(nk)}$$

and the number of states with relative complexity $C = \frac{nm}{2}$ is

$$d \cdot (d-1)^{m-1} \approx d^m = (nk)^{\mathcal{C}} = e^{\mathcal{C} \log(nk)}.$$

Next, let us discuss this result from both wiew points: $e^{\mathcal{C} \log(nk)}$ being the number of all the states and $e^{\mathcal{C} \log(nk)}$ being the number of operators requiring circuits consisting of $C = \frac{nm}{2}$ gates.

First, in some sense, the number of all the possible states is the volume of configuration space. The number of states grows exponentially with the allowed relative complexity:

$$V(\mathcal{C}) = e^{\mathcal{C} \log(nk)},$$

therefore, the relative complexity is propotional to the logarithm of the number of states

$$\mathcal{C} = \frac{1}{\log(nk)} \log V.$$

For another key observation we pick any random state on the tree as a starting point. Now, if we were to perform a random walk, move with some depth-1 allowed circuit, we would almost never walk toward the root, always outward as d is typically very large. Relative complexity has strong tendecy to grow. All things considered, this sugests that relative complexity is some kind of entropy.

Second, observe that the number of states (or, alternatively, operators/circuits) with the given complexity $\mathcal{C}$ grows exponentially with $\mathcal{C}$. *But where is the limit? What is the maximal possible relative complexity?* The tree must eventually produce all the $\frac{1}{\varepsilon^{2N-2}}$ states on the $CP(N-1)$, so,

$$\begin{aligned} \frac{1}{\varepsilon^{2N-2}} &\stackrel{!}{=} (nk)^{\mathcal{C}_{\max}}. \\ (2N-2)\log\frac{1}{\varepsilon} &= \mathcal{C}_{\max}\log(nk) \\ \mathcal{C}_{\max} &= \frac{(2\cdot 2^n - 2)\log\frac{1}{\varepsilon}}{\log(nk)} \approx 2^n \log\frac{1}{\varepsilon}. \end{aligned}$$

To conclude, the maximal distance between two states $|A\rangle$ and $|B\rangle$ is $\mathcal{C}_{\max} \approx 2^n \log\frac{1}{\varepsilon}$. As expected, there exist circuits that require exponential circuit depth. But the fact that the number of circuits of depth exactly $\mathcal{C}$ grows exponentially with $\mathcal{C}$ implies that the majority of circuits require exponential depth. Therefore, exponentially many quantum states require exponential circuit depth to be synthesized.

For further study, we recommend [68, 13, 14], which also inspired this section. Moreover, considering $\mathcal{G}$ exactly universal there exists a metric on $SU(2^n)$ such that the distance between the identity operator and the operator $U \in SU(2^n)$ we wish to synthesize (the length of the minimal geodesic between $\mathbb{I}$ and U) poses a lower bound on the minimal number of gates from $\mathcal{G}$ required to exactly synthesize U [55]. For $\mathcal{G}$ densely generating $SU(2^n)$,[2] Solovay-Kitaev's algorithm [19] produces a sequence of $\mathcal{O}(\log^{3.97}(\frac{1}{\epsilon}))$ quantum gates that is guaranteed to approximate the desired quantum gate to an accuracy within $\epsilon > 0$.

So far, we have discussed general transitions between states and their complexity. What will the situation be like if we pose a restriction on the allowed energy of the system? This will be the topic of the next section.

[2]Given any $U \in SU(2^n)$ and any accuracy $\epsilon > 0$ there exists a product $g_1 g_1 \dots g_m$ of gates from $\mathcal{G}$ which is an ϵ-approximation to U: $\sup_{|\psi|=1} |(U - g_1 g_1 \dots g_m)|\psi\rangle| < \epsilon$.

1.2 GSCON: Ground state connectivity

In 2019, Sevag Gharibian and Jamie Sikora [32] asked: *Given two ground states $|\psi\rangle$ and $|\phi\rangle$ of a local Hamiltonian H, are $|\psi\rangle$ and $|\phi\rangle$ connected through the ground space of H or are they sepparated by an "energy barrier"?* Let us formalize the definition of this problem, for our purposes, in the following way:

Definition 1.1 (Ground state connectivity (GSCON))): *Given as input:*

- *k-local Hamiltonian $H = \sum_i H_i$, i.e., an n-qubit Hermitian operator H that can be decomposed into a sum of Hamiltonians H_i each of which acts upon at most k qubits;*
- *polynomial size quantum circuits U_ψ and U_ϕ generating "starting" and "target" ground states of H, respectively. That is, $U_\psi |0\rangle^{\otimes n} = |\psi\rangle$, $U_\phi |0\rangle^{\otimes n} = |\phi\rangle$, and $\langle\psi| H |\psi\rangle = \langle\phi| H |\phi\rangle = E_0$.*
- *Integers $m, l \geq 1$;*
- *real numbers E_{max} and $\varepsilon \geq 0$.*

Does there exist a sequence of l-local unitaries $(U_i)_{i=1}^m$ such that

- *$|\psi\rangle$ is mapped to $|\phi\rangle$:*

$$\left|U_m \dots U_1 |\psi\rangle - |\phi\rangle\right| = \sqrt{\sum_{i=0}^{2^n-1} (U_m \dots U_1 |\psi\rangle - |\phi\rangle)_i^2} \leq \varepsilon.$$

- *All intermediate states remain in the low energy space. That is, for all $i \in \{1, \dots, m\}$, the intermediate state $|\psi_i\rangle := U_i \dots U_1 |\psi\rangle$ satisfies $\langle\psi_i| H |\psi_i\rangle \leq E_{max}$.*

This problem turns out to be physically relevant due to its connection to *quantum memories* and *stabilizer codes*. These areas attempt the "holy grail" in building quantum computers – the ability to store quantum information and perform fault-tolerant computation. A common approach encodes the qubits' state into a degenerate ground space of a gapped Hamiltonian, adding large redundancy (i.e., encoding k qubits into $n > k$ qubits) in the process. Assuming errors occur on a small number of qubits at a time, it should ideally not be possible for one ground state $|\psi\rangle$ to be mapped to another ground state $|\phi\rangle$ through the low energy space via a sequence of local operations. Along these lines, let us look at Kitaev's Toric code and a simple Ising memory for illustration.

Kitaev's Toric Code

Kitaev's Toric Code, introduced by Alexei Kitaev in 2003 [51], is a topological quantum error-correcting code and an example of a stabilizer code. It encodes two qubits into N qubits placed on edges of a square lattice on the torus. The four computational basis states $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$ are in one-to-one correspondence with the four ground states of

$$H_{\mathrm{TC}} = -\sum_{v\in\text{vertices}} A_v - \sum_{p\in\text{plaquettes}} B_p.$$

The operators

$$A_v := \prod_{j\in\mathrm{star}(v)} X_j, \qquad B_p := \prod_{j\in\mathrm{boundary}(p)} Z_j.$$

form the stabilizer of the code. They generate the set of operators that do nothing to the codewords. A great introductory material to stabilizer codes and quantum error correction is Gottesman's [34].

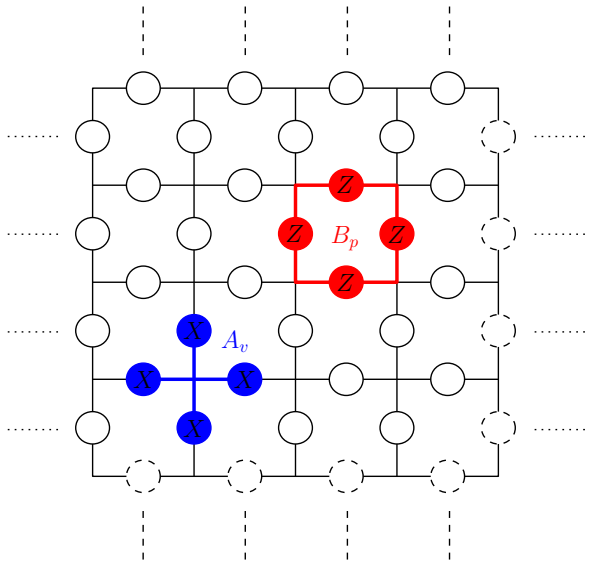
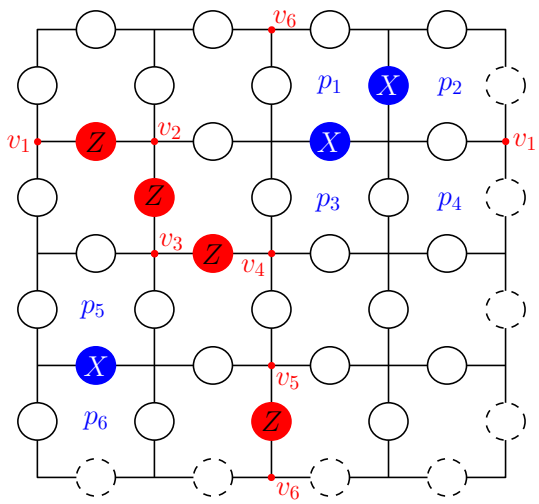


Figure 1.4: A 32-qubit example of Kitaev's Toric code. Stabilizer is made of N operators, $\frac{N}{2}$ A_v operators for each vertex, and $\frac{N}{2}$ B_p operators for each plaquette.

The code space of any stabilizer code is the $+1$ eigenspace of the stabilizer $\mathcal{S}$. The Hamiltonian H_{SC} for stabilizer code is constructed as a negative sign sum of all the operators from the stabilizer, so that the ground space of H_{SC} is identical with the $+1$ eigenspace of $\mathcal{S}$. As the effect of any $S \in \mathcal{S}$ on the codeword $|\psi\rangle$ is trivial, $S|\psi\rangle = |\psi\rangle$, when measuring S, we expect outcome $+1$. Measuring all elements of the stabilizer is the typical approach to detect errors.

To illustrate, cosider the following errors in our toric code:



In this case, the set of operators $\{A_{v_1}, A_{v_4}, A_{v_5}, A_{v_6}, B_{p_2}, B_{p_3}, B_{p_5}, B_{p_6}\}$ detects there are some errors. A_{v_1}, A_{v_4}, A_{v_5}, and A_{v_6} tell us there is an odd number of Z errors on the qubits they check, B_{p_2}, B_{p_3}, B_{p_5}, and B_{p_6} tell us there is an odd number of X errors on the qubits they check. Notice that we detect only endpoints of the errors. The nontrivial task in the error correction is to match the endpoints correctly.

A convenient property of Kitaev's toric code for error correction is that all contractible loops are products of stabilizer operators, and therefore they act on the codewords trivially. For that reason, if we match the endpoints correctly, even if we guess a wrong path, the actual path together with the wrong path form a loop, and if the loop is contractible, which it almost certainly is, the error is corrected.

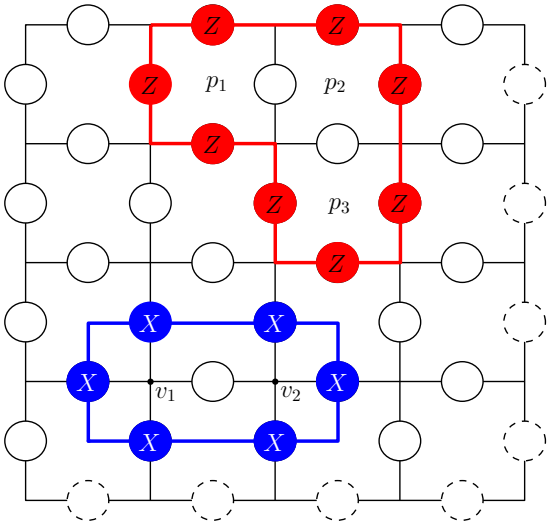


Figure 1.5: Contractible loops of X or Z operators are products of stabilizer operators, e.g., the blue loop is $A_{v_1} \cdot A_{v_2}$ and the red loop $B_{p_1} \cdot B_{p_2} \cdot B_{p_3}$. Therefore, they are undetectable and have no effect on the stored quantum state.

However, complementing the errors into non-contractible loops poses a serious threat. bThat is because non-contractible loops of X or Z operators act as logical

operators on the codewords.

H_{TC} exhibits topological order as its ground states cannot be distinguished or connected by a local operator. In fact, they are connected via non-contractible loops of X operators. There are two such loops on the torus, and we also expect two logical NOT operators $\overline{X}_1$ and $\overline{X}_2$, one for each of the two encoded qubits.

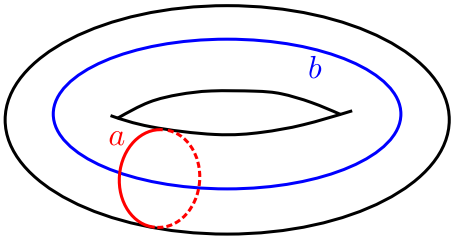
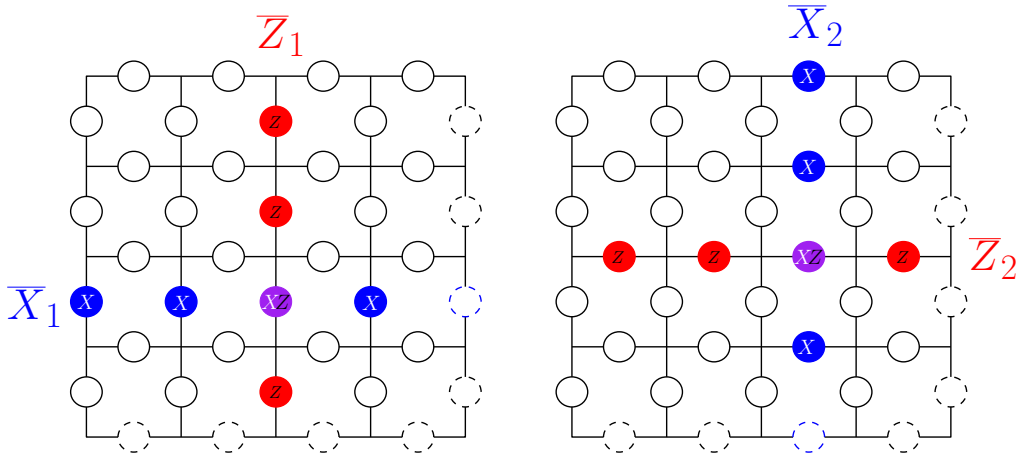


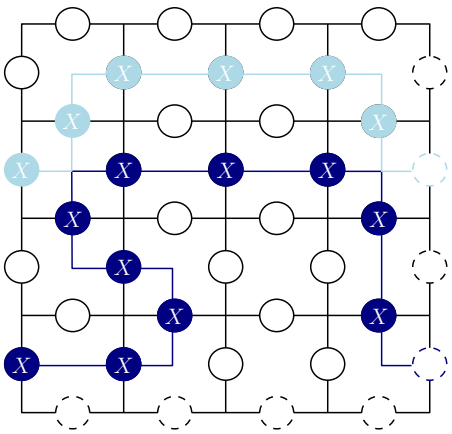
Figure 1.6: There are two types of non-contractible loops on the torus: a and b.

To be specific, the logical operators $\overline{X}_1$, $\overline{Z}_1$ and $\overline{X}_2$, $\overline{Z}_2$ are defined in the following way:



Observe that all these operators commute with the stabilizer $\{A_v, B_p\}$. Of course, X_i anticommutes with Z_i as expected.

However, the depicted logical operators are not the only possibilities. There are many more representations. For instance, the following are also valid $\overline{X}_1$ operators:



As mentioned, all the non-contractible loops of either X operators or Z operators will have the desired effect on the encoded quantum state $|\psi\rangle$ (which qubit the operator acts on is determined by the direction (top-bottom/left-right)/type (a or b) of the loop). But we need to restrict one of the operators, here we choose $\overline{X}$, to purely cross the edges, and the other one, here we select $\overline{Z}$, to follow the edges in order for $\overline{X}_i$ and $\overline{Z}_i$ to anticommute in a manner they are supposed to. Moreover, as contractible loops are products of stabilizer operators, $\overline{X}_i\overline{X}_i = \mathbb{I}$ and $\overline{Z}_i\overline{Z}_i = \mathbb{I}$ regardless of whether the representations are identical.

Of course, there is much more to say about Kitaev's toric code. Our goal here was only to motivate the discussion about its tolerance toward errors. The only nondetectable errors are logical operators and contractible loops, and the logical operators are exactly the case we try to avoid. But as we have already mentioned, the logical operators are nonlocal. Under the assumption that the errors act locally, we expect the probability of such a thing happening almost identical to zero. Nevertheless, we believe it would be helpful to understand the details. How tough would it be to compose the logical NOT from local operators? Considering we could detect if the system's energy exceeded some value, what should this value be to avoid any logical error being undetected? How often should the code be checked for errors? We believe that solving GSCON on stabilizer codes might bring valuable insights, not just from the complexity theory view. Next, for illustration, let us introduce and solve a very simple Ising-memory-inspired GSCON instance.

Ising memory

The Ising model consists of spins arranged in a graph (usually a lattice). The graph determines which spins are allowed to interact with each other. The energy of the spin-system is given by

$$H_{\text{Ising}} = -\sum_{\langle i,j\rangle\in\text{edges}} J_{ij}Z_iZ_j - \sum_{j\in\text{spins}} h_jZ_j,$$

where J_{ij} determines the strength of the spins' interactions, as well as whether the interacting spins prefer to disagree ($J_{ij} < 0$ dictates antiferromagnetic interactions) or agree ($J_{ij} > 0$ gives ferromagnetic interactions), and h_j describes the external field ($h_j = 0 \implies$ no external influence, $h_j > 0 \implies$ the jth spin prefers to occupy the state $|\uparrow\rangle \equiv |0\rangle$, $h_j < 0 \implies$ the jth spin desires being in the state $|\downarrow\rangle \equiv |1\rangle$).

For the simple self-correcting Ising memory, we will consider the ferromagnetic case with all $J_{ij} = 1$, and no external field ($h = 0$). The system is then described by the Hamiltonian

$$H = -\sum_{\langle i,j\rangle\in\text{edges}} Z_iZ_j,$$

which is minimised when all the spins are equal. So, our Ising lattice has two energy minima:

- $|\bar{0}\rangle = |00\ldots0\rangle$ – all spins in the state $|0\rangle$,
- $|\bar{1}\rangle = |11\ldots1\rangle$ – all spins in the state $|1\rangle$.

Unfortunately, the Ising memory as a stabilizer code with stabilizer $\{Z_iZ_j\}_{\langle i,j\rangle\in\text{edges}}$ is not fault tolerant toward Pauli Z errors, as they commute with the stabilizer. However, it can serve well as a classical memory. We will encode a state of a single classical bit into this system. The fault tolerance comes from majority voting. After some time, we measure the spins to restore the stored bit's value to the majority vote.

If one measures the spins often enough, no local errors should go undetected. The problem appears if more than half of the qubits is flipped. Suppose we can detect if more than $E_{\max}$ energy is poured into the system. It can't hurt to know how fast could a majority 0 be locally transformed to a majority 1, and whether it could happen undetected. At least, we would gain some insights about the system.

For illustration, let us consider the simplest examle: the two-qubit case. Of course, this system is too small for the majority vote to be useful, but it is illustrative for GSCON. The Hamiltonian is $H = -Z_1Z_2$, and we are interested in the transition from the ground state $|00\rangle$ to the ground state $|11\rangle$. There are only two cases to consider: l, the locality of allowed operators, being 1 and $l = 2$. If 2-qubit operators are allowed, there's nothing to discuss as $X_1X_2\,|00\rangle = |11\rangle$. In case of 1-local operators, our goal is to find a sequence of single-qubit gates $g_1, g_2, \ldots, g_m$, such that

$$g_1g_2\ldots g_m = X_1X_2$$

up to the global phase.

A general pure single-qubit state $|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle$ can be represented as a point on the Bloch sphere. Unitary operators correspond to rotations in $\mathbb{R}^3$. Any such rotation can be composed as a sequence of rotations around two fixed axes. We choose

$$\mathcal{G} = \left\{e^{i\alpha P} | P \in \{X_1, Z_1, X_2, Z_2\}, \alpha \in [0, 2\pi]\right\}.$$

All $e^{i\alpha Z_1}$ and $e^{i\alpha Z_2}$ influence only the relative phases and have no effect on the energy of the system. They commute with the Hamiltonian. Therefore, we can restrict ourselves to Pauli X rotations. Our new goal is to find a set of angles $\{(x_j, y_j)\}_{j=1}^{m/2}$ such that

$$\prod_{\{(x_j,y_j)\}_{j=1}^{m/2}} e^{ix_jX_2}e^{iy_jX_1}\,|00\rangle = |11\rangle$$

up to the global phase. Each intermediate state can be written as

$$|\psi\rangle_k = e^{ixX_2}e^{iyX_1}\,|00\rangle$$

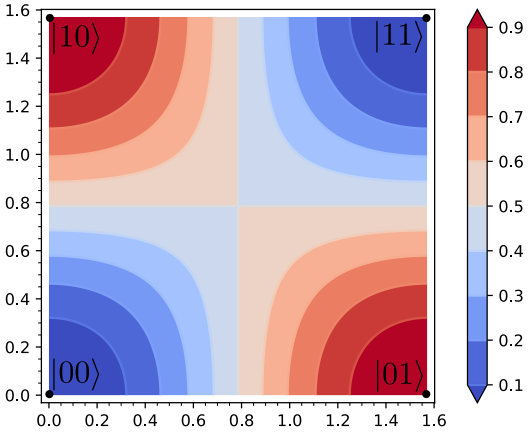
for $x = \sum_{j=0}^{k/2} x_j$ and $y = \sum_{j=0}^{k/2} y_j$.

$$|\psi\rangle_k = e^{ixX_2} e^{iyX_1} |00\rangle =$$
$$= \cos x \cos y |00\rangle + i \cos y \sin x |01\rangle + i \cos x \sin y |10\rangle - \sin x \sin y |11\rangle .$$

The energy of $|\psi\rangle_k$ is

$$E(|\psi\rangle_k) = \cos^2 y \sin^2 x + \cos^2 x \sin^2 y.$$

Let us plot this energy for all viable intermediate states $\psi(x, y)$ in a contour plot:



$e^{iy_j X_1}$ moves the state in the vertical direction in the graph, and $e^{ix_j X_2}$ in the horizontal direction. Therefore, we are limited to moves in the purely horizontal and purely vertical direction. As the energy in the red squares is greater than or equal to 0.5, at some point an intermediate state must have energy at least 0.5. We also know there exists such a path, for example,

$$e^{i\frac{\pi}{4}X_1} e^{i\frac{\pi}{2}X_2} e^{i\frac{\pi}{4}X_1} |00\rangle = -|11\rangle .$$

2 Algorithms for NISQ devices

Being restricted to a relatively small amount of qubits (10-1000) and, at best, error mitigation presents a tremendous obstacle, the true challenge, in devising useful NISQ algorithms. At the moment, it seems the best we can do is to define a parametrized family of algorithms, and one then attempts to solve a chosen optimization problem by finding the algorithm within the family which is best at solving the problem. Therefore, the most promising applications for NISQ devices are *Quantum Machine Learning (QML)*, *the Quantum Approximate Optimization Algorithm (QAOA)*, and *the Variational Quantum Eigensolver (VQE)*. In this chapter, we first briefly review QML, then take a closer look at QAOA. For further reading, we recommend [9].

2.1 QML: Quantum Machine Learning

Very roughly said, the goal of machine learning is to create a model fitting data samples that can be more or less reliably applied to new data. To illustrate, consider the task of assigning correct labels to data points x. The data points follow a probability distribution $p(x)$. We assume there exists some "ground truth", a function $f^*(x)$ from all viable data points to correct labels.

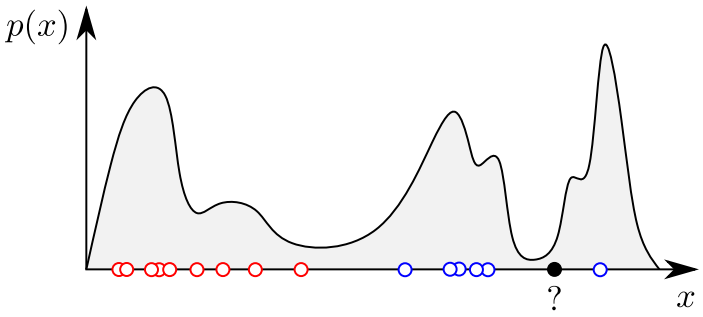


Figure 2.1: An example with two labels: *red* and *blue*. Consider the red and blue points to be all the data samples we have. Based on these data samples, we form a model (e.g., a line separating the red and the blue dots) for labeling data points we haven't seen yet, e.g., what color should the black dot have (located left or right to the line)?

We try to model $f^*(x)$ (unknown to us) by a class of models/hypotheses $f(x)$. We measure a model $f(x)$'s distance from the ground truth by a loss function $l(f^*(x), f(x))$. Our goal is to find

$$f_{\text{opt}} = \min_f \int p(x) l(f^*(x), f(x)) \mathrm{d}x.$$

Depending on the nature of the task, machine learning is traditionally divided into the following three areas.

- **Supervised learning.** The computer is presented with example inputs and their desired outputs to learn from. The goal is to learn a general rule that maps inputs to outputs. The two prominent methods to perform a supervised learning classification task using a NISQ computer are *quantum kernel estimation* [40, 42, 52, 64] and *Variational Quantum Classifier* [28, 54]. Regarding Chapter 3, let us mention that also a *Gaussian BosonSampling* device can be used for computing kernel functions [63].

- **Unsupervised learning.** In this setting, the computer is left on its own to find structure in the provided input. Simply stated, the task is to recognize an underlying pattern in the data. There are quantum algorithms for generative modelling (learning the underlying probability distribution from a finite set of samples from a data set, and generating new samples from the distribution) [7], for clustering (dividing data points into a number of groups such that data points in the same groups are more similar to other data points in the same group than to those in other groups) [58], and more [9].

- **Reinforcement learning.** A computer program (an "agent") interacts with a dynamic environment attempting to maximize an underlying reward function. By harnessing quantum mechanical phenomena such as superposition and entanglement, one can expect to achieve speedups in the reinforcement learning tasks [20, 22, 59, 23].

(Quantum) Machine Learning is too broad an area with very active research to provide even the slightest chance to condense an overview to only a few pages. However, we felt an obligation to try to do so as the machine learning area, by its spirit, is very close to the NISQ era, not just by being its main application but also by the concept: "Let's try it and see what happens". With the performance analysis of NISQ algorithms being so demanding, it seems the best approach is to (first) study the performance on typical instances practically. For the remainder of this chapter, let us look at one such algorithm.

2.2 QAOA: The Quantum Approximate Optimization Algorithm

QAOA [26, 25] is a hybrid classical-quantum algorithm developed for finding approximate solutions to *combinatorial optimization problems*.

2.2.1 Combinatorial Optimization Problems (COPs)

COPs are characterized by a set of m constraints $\{C_\alpha\}_{\alpha=1}^m$, which are binary value functions on n-bit strings $z = z_1 z_2 \dots z_n \in \{0,1\}^n$:

$$C_\alpha(z) = \begin{cases} 1 & \text{if } z \text{ satisfies the constraint } C_\alpha \\ 0 & \text{otherwise.} \end{cases}$$

The goal is then to find a string z that maximizes the *objective function*

$$C(z) = \sum_{\alpha=1}^{m} C_\alpha(z).$$

This, however, often turns out to be a very resource-consuming problem. We might need to check all the 2^n possible solutions to be sure we have indeed found the maximum.[1] In many cases, to save or having not enough resources, we might be willing to settle for a *good enough* solution. Our new objective would be to find a solution z that satisfies a fraction of the constraints close to the maximum $C_{\max}$,

$$C_{\max} := \max_z C(z).$$

Besides the running time, an algorithm for finding approximate solutions is characterized by an *approximation ratio* $r^\star$ defined as the lower bound on the fraction of constraints which the outputed string promises to satisfy:

$$C(z) \geq r^\star \cdot C_{\max}.$$

To get a better understanding, let us take a look at the Maximum Cut (MaxCut) problem and introduce several approximation algorithms for finding "good enough" MaxCut solutions.

[1] For illustration, look at the greatest n for which a program performing 2^n steps finishes its computation for various runtimes:

runtime	millisecond	second	minute	hour	day	month	year	millennium
greatest n	20	30	35	41	46	51	54	64

Of course, the real values depend on the power of the computer. This table is only illustrative. Considering the task fully parallelizable, then, for example, we would need 60 such computers to finish $n = 41$ computation in one minute, 24 computers to finish $n = 46$ computation in one hour, $60 \cdot 24 = 1440$ computers to finish the $n = 46$ computation in one minute, 365 computers to finish $n = 54$ computation in one day.

MaxCut: The Maximum Cut problem

Given a graph

$$G = (V - \text{set of vertices}, E - \text{set of edges}),$$

partition the vertices into two sets (color/label the vertices with black and white/0 and 1) in a way that maximizes the *cut*, i.e., the number of edges connecting these two sets.[2]

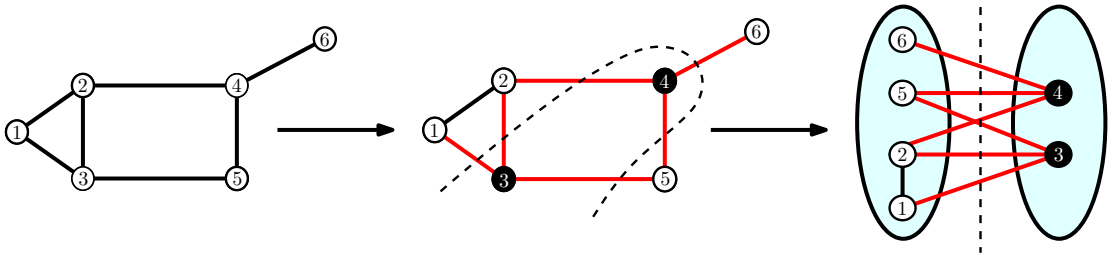


Figure 2.2: A simple instance of the MaxCut problem. The image depicts solution $z = 001100$. Solutions 101100, 110011, and 010011 are also optimal. The maximum cut value is 6. The graph is not bipartite; there must be at least one unsatisfied edge.

You can find this problem in physics when looking for a ground state of an antiferromagnetic Ising model. Antiferromagnetic interaction energetically favors opposite magnetizations for neighboring spins. This is simple on bipartite graphs. However, in general, such systems are frustrated, and finding an optimal state can be difficult.

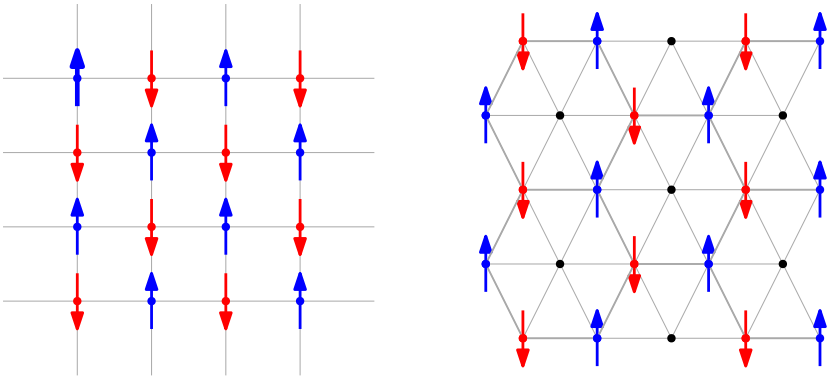


Figure 2.3: On the left: a ground state of a square-lattice antiferromagnetic Ising model. On the right: a ground state of a triangular-lattice antiferromagnetic Ising model. In the triangular-lattice case, the black spins' values do not affect the maximum cut value, thus, we can choose their values arbitrarily.

A very simple approximation algorithm for MaxCut would be the following:

[2] A more general formulation gives each edge a weight and the cut is defined as the sum of the weights of the edges joining the two sets. In this thesis, we will consider only the case when all edges have the same weight (1).

```
#Partition the vertices arbitrarily: V → (A, B).
repeat (until there is nothing to do):
    for each vertex:
        if less than half of its edges cross the cut:
            move the vertex to the other side of the cut
return the final cut (A,B)
```

The approximation ratio of this algorithm is guaranteed to be 1/2 as the final cut is crossed by at least half of the edges ($|E|/2$) and the maximal cut cannot be greater than the number of all edges $|E|$. What about its time complexity? The number of iterations is at most $|E|$ because each iteration improves the cut by at least 1. Each iteration takes $O(|E| \cdot |V|)$ time as it consists of a for-loop through all the vertices, where, for each vertex, one must look at no more than $|E|$ edges. Thus, the overall time complexity is $O(|E|^2 \cdot |V|)$ and, therefore, polynomial.

Many approximate algorithms leverage randomness to simplify both the runtime and the analysis. For illustration, consider a simple "random guessing" algorithm for MaxCut:

```
Return a random (p_0 = p_1 = 0.5) |V|-bit string as a candidate solution.
```

This algorithm performs solely $\mathcal{O}(|V|)$ steps. The approximation ratio is, for simplicity, taken to correspond to the average case. For each edge, the two connected vertices are equally likely to agree in their value as they are to disagree. Therefore, each edge has a 50/50 chance to be included in the cut. Hence, in the average case, the cut crosses half of the edges. The number of edges is a theoretical maximum for MaxCut, so whatever the optimal value of MaxCut is, random guessing should achieve at least 1/2 of its value. The approximation ratio is thus lower bounded by $r^\star = 0.5$. So, we obtained simplicity but lost the worst-case guarantee.

The best known polynomial-time approximation algorithm for MaxCut, Goemans-Williamson [33], has an approximation ratio $r^\star \approx 0.878$. It is known to be NP-hard to achieve $r^\star$ better than $\frac{16}{17} \approx 0.941$ [71, 45]. Moreover, if the Unique Games Conjecture (UGC)[3] [48] holds, then one cannot do better than Goemans-Williamson [49] in polynomial time. Therefore, Farhi, Goldstone, and Gutmann decided to take a look at MaxCut and, in 2014, invented a quantum algorithm for approximate optimization [26] with an optimistic goal to shed light on (and possibly disprove) the UGC, (hopefully) clarifying the relationship between NP, approximations for NP, and the quantum

[3]The UGC roughly says that the best approximations for hard problems are achieved by semidefinite programming, and anything better would basically result in solving NP-complete problems efficiently, which is unlikely.

complexity class BQP in the process. The algorithm has not yet succeeded at that goal but has opened an interesting discussion, motivated several classical algorithms, and basically started a whole industry of experiments on few-qubit quantum systems. So, let us look at this new quantum approach to approximate combinatorial optimization.

2.2.2 QAOA

We can encode any combinatorial optimization problem, fully characterized by objective function $C = \sum_{\alpha=1}^{m} C_\alpha$, into a Hamiltonian $H_C = \sum_{\alpha=1}^{m} H_{C_\alpha}$ diagonal in the computational basis:

$$H_C |z\rangle = \sum_{\alpha=1}^{m} H_{C_\alpha} |z\rangle = \sum_{\alpha=1}^{m} \Big(1 - C_\alpha(z)\Big) |z\rangle = \Big(m - C(z)\Big) |z\rangle .$$

The ground state of H_C is the optimal solution.

In approximate optimization, our goal is to find a good enough solution for C. Therefore, in the quantum setting, we aspire to create a low energy state of the (problem) Hamiltonian H_C. In p-level QAOA (or QAOA$_p$), we try to accomplish this by constructing a parameter-dependent quantum state

$$\left|\vec{\gamma}, \vec{\beta}\right\rangle = \underbrace{e^{-i\beta_p H_M} e^{-i\gamma_p H_C}}_{p\text{th round}} \cdots \underbrace{e^{-i\beta_2 H_M} e^{-i\gamma_2 H_C}}_{\text{2nd round}} \underbrace{e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C}}_{\text{1st round}} |s\rangle , \tag{2.1}$$

where the parameters are:

- integer $p \geq 1$ determining the number of rounds,
- $2p$ angles $\gamma_1\gamma_2 \ldots \gamma_p \equiv \vec{\gamma}$ and $\beta_1\beta_2 \ldots \beta_p \equiv \vec{\beta}$ specifying times for the alternating unitary evolutions governed by Hamiltonians H_C and H_M, respectively.

In the original paper [26], the initial state $|s\rangle$ was proposed to be

$$|s\rangle = |+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle ,$$

the ground state of

$$H_{\mathrm{M}} = -\sum_{j=0}^{n} X_j .$$

If there were to appear a good reason, $|s\rangle$, H_{M}, or H_{C} could be modified, even round-dependent. Anyhow, $|s\rangle$ is required to be easy to prepare. Here we will consider purely the original setting. After the state $\left|\vec{\gamma}, \vec{\beta}\right\rangle$ is prepared, we measure it in the computational basis to obtain a string z and evaluate $C(z)$ (the objective function). The hope is that multiple rounds of alternating these Hamiltonians produce a string z, which gives $C(z)$ close to the maximum.

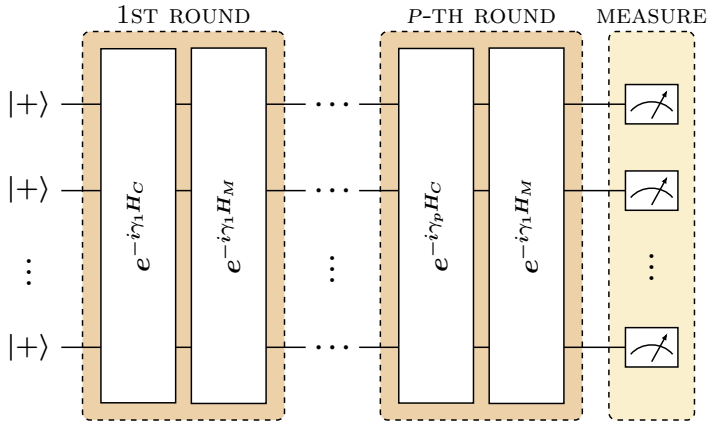


Figure 2.4: A crucial subprocess of QAOA. The preparation of the state $\left|\vec{\gamma},\vec{\beta}\right\rangle$ and the subsequent computational basis 0/1 measurement to get a candidate for the near-optimal solution.

The first (key and very challenging) part is choosing the right parameters. Our goal is to create a low energy state of the (problem) Hamiltonian H_C. It means that we seek such parameter for which the average energy

$$F_p(\vec{\gamma},\vec{\beta}) = \left\langle \vec{\gamma},\vec{\beta}\right| H_C \left|\vec{\gamma},\vec{\beta}\right\rangle \tag{2.2}$$

is minimized, and thus $C(z)$ on the strings we obtain large. Let us denote

$$M_p = \min_{\vec{\gamma},\vec{\beta}} F_p(\vec{\gamma},\vec{\beta}). \tag{2.3}$$

As the minimization for the $(p-1)$-level algorithm can be viewed as a constrained minimization for the p-level algorithm with $\beta_p = \gamma_p = 0$, M_p can only decrease as p grows:

$$M_{p-1} \geq M_p. \tag{2.4}$$

Furthermore, the Suzuki-Trotter formula [69],

$$e^{A+B} = \lim_{n\to\infty} \left(e^{A/n} e^{B/n}\right)^n, \tag{2.5}$$

guarantees that there exists (in a $p \to \infty$ limit) a QAOA algorithm, which is equivalent to the adiabatic evolution from the ground state of H_{M} to the ground state of H_C. Thanks to the adiabatic theorem [10, 15, 21]

Theorem 1 (The adiabatic theorem[4]): *A physical system remains in its instantaneous eigenstate if a given perturbation is acting on it slowly enough and if there is a gap between the eigenvalue and the rest of the Hamiltonian's spectrum.[10]*

the adiabatic evolution promises optimal results for large enough T. It follows that

$$\lim_{p\to\infty} M_p = \min_z C(z). \tag{2.6}$$

Recently, it was shown that QAOA is at least counteradiabatic, not just adiabatic, and has better performance than finite time adiabatic evolution [75]. However, in the NISQ era, we are interested in small, at most polynomial, number of rounds and there's no known reason why imitating any adiabatic/counteradiabatic schedule should be the best method. So far, there's no shortcut. To find the optimal angles, we need to examine all the $(2\pi \times 2\pi)^{2p}$ parameter landscape. There are many methods to choose from for this search, from reinforcement learning [47, 73], multi-start methods [65], to simple grid search or gradient descent, each having its pros and cons.

Once we have the optimal parameters, we should keep in mind, that the state $\left|\vec{\gamma},\vec{\beta}\right\rangle$ we prepared probably isn't a superposition of purely optimal and good enough solutions. Therefore, once the parameters were optimized, we repeat the process – the preparation of the state $\left|\vec{\gamma},\vec{\beta}\right\rangle$, computational basis 0/1 measurement, evaluating the objective function on the measurement result – many times to ensure we find at least as good solution as we expect.

Locality of QAOA

Let us consider QAOA applied to MaxCut. The respective Hamiltonian is

$$H_{\mathrm{C}} = \frac{1}{2}\sum_{\langle k,l\rangle\in E} \left(\mathbb{I} + Z_k Z_l\right).$$

Given any set of parameters, the expected result from QAOA (the mean energy) is

$$F_p(\vec{\gamma},\vec{\beta}) = \left\langle\vec{\gamma},\vec{\beta}\right| H_C \left|\vec{\gamma},\vec{\beta}\right\rangle = \frac{1}{2}\left(1 + \sum_{\langle k,l\rangle\in E} \left\langle\vec{\gamma},\vec{\beta}\right| Z_k Z_l \left|\vec{\gamma},\vec{\beta}\right\rangle\right). \tag{2.7}$$

Let us study the graph-dependent terms:

$$F^p_{\langle kl\rangle}(\vec{\gamma},\vec{\beta}) := \left\langle\vec{\gamma},\vec{\beta}\right| Z_k Z_l \left|\vec{\gamma},\vec{\beta}\right\rangle =$$
$$= \langle s|\underbrace{e^{i\gamma_1 H_C} e^{i\beta_1 H_{\mathrm{M}}}}_{1\text{th round}} \ldots \underbrace{e^{i\gamma_p H_C} e^{i\beta_p H_{\mathrm{M}}}}_{p\text{th round}} Z_k Z_l \underbrace{e^{-i\beta_p H_{\mathrm{M}}} e^{-i\gamma_p H_C}}_{p\text{th round}} \ldots \underbrace{e^{-i\beta_1 H_{\mathrm{M}}} e^{-i\gamma_1 H_C}}_{1\text{st round}} |s\rangle.$$

Specifically, let us proceed from the inside out, and observe that this term depends only on the p-neighborhood of the edge $\langle k,l\rangle$. The innermost part is

$$\underbrace{e^{i\gamma_p H_C} e^{i\beta_p H_{\mathrm{M}}}}_{p\text{th round}} Z_k Z_l \underbrace{e^{-i\beta_p H_M} e^{-i\gamma_p H_C}}_{p\text{th round}}.$$

[4] Here stated in its original form (due to M. Born and V. Fock (1928)). Later analyses added formal guarantees of convergence which involve $||H||$, $\left|\left|\frac{\partial H}{\partial t}\right|\right|$, $\left|\left|\frac{\partial^2 H}{\partial t^2}\right|\right|$, the spectral gap, etc. If interested in a better formulation/analysis, you might take a look at [15] or [21], for example.

As $e^{-i\beta_p H_M} = e^{i\beta_p \sum_{j=0}^{n} X_j} = \prod_{j=0}^{n} e^{i\beta_p X_j}$,

$$e^{i\beta_p H_M} Z_k Z_l e^{-i\beta_p H_M} = \left(\prod_{a=0}^{n} e^{-i\beta_p X_a}\right) Z_k Z_l \left(\prod_{b=0}^{n} e^{i\beta_p X_b}\right).$$

All terms $e^{-i\beta_p X_a}$ that don't affect qubits k and l commute through and disappear into identity. Analogically, we can let all terms in $e^{-i\gamma_p H_C} = e^{-i\gamma_p \sum_{\langle a,b\rangle \in E} (\mathbb{I} + Z_a Z_b)} = e^{-i|E|\gamma_p} \prod_{\langle a,b\rangle \in E} e^{-i\gamma_p Z_a Z_b}$ that don't affect qubits k and l commute through and disappear. We get

$$\underbrace{e^{i\gamma_p H_C} e^{i\beta_p H_M}}_{p\text{th round}} Z_k Z_l \underbrace{e^{-i\beta_p H_M} e^{-i\gamma_p H_C}}_{p\text{th round}} = e^{i\gamma_p H_C} \left(e^{-i\beta_p X_l} e^{-i\beta_p X_l} Z_k Z_l e^{i\beta_p X_k} e^{i\beta_p X_l}\right) e^{-i\gamma_p H_C} =$$

$$= \prod_{\langle a,b\rangle \in g(\langle k,l\rangle + \text{distance-1})} e^{i\gamma_p Z_a Z_b} \cdot e^{-i\beta_p X_l} e^{-i\beta_p X_l} Z_k Z_l e^{i\beta_p X_k} e^{i\beta_p X_l} \cdot \prod_{\langle a,b\rangle \in g(\langle k,l\rangle + \text{distance-1})} e^{-i\gamma_p Z_a Z_b}.$$

This term already affects not only qubits k and l but also all qubits at distance 1 from them via terms $e^{-i\gamma_p Z_{j\in\{k,l\}} Z_{\text{distance-1}}}$ which cannot commute through. If we continued and wrapped the innermost term into depth-$(p-1)$ evolutions, we could observe that the 2-round innermost term

$$\underbrace{e^{i\gamma_{p-1} H_C} e^{i\beta_{p-1} H_M}}_{(p-1)\text{th round}} \underbrace{e^{i\gamma_p H_C} e^{i\beta_p H_M}}_{p\text{th round}} Z_k Z_l \underbrace{e^{-i\beta_p H_M} e^{-i\gamma_p H_C}}_{p\text{th round}} \underbrace{e^{-i\beta_{p-1} H_M} e^{-i\gamma_{p-1} H_C}}_{(p-1)\text{th round}}$$

depends on qubits k and l and those qubits that are at most two steps from qubits k and l. With each round, the relevant subgraph – the part of the graph that affects $F^p_{\langle kl\rangle}(\vec{\gamma}, \vec{\beta})$ – increases by depth one. $F^p_{\langle kl\rangle}(\vec{\gamma}, \vec{\beta})$, and so the value of qubits k and l, depends only on the subgraph involving qubits k and l and those at a distance no more than p away. This is true for any problem, not just MaxCut.

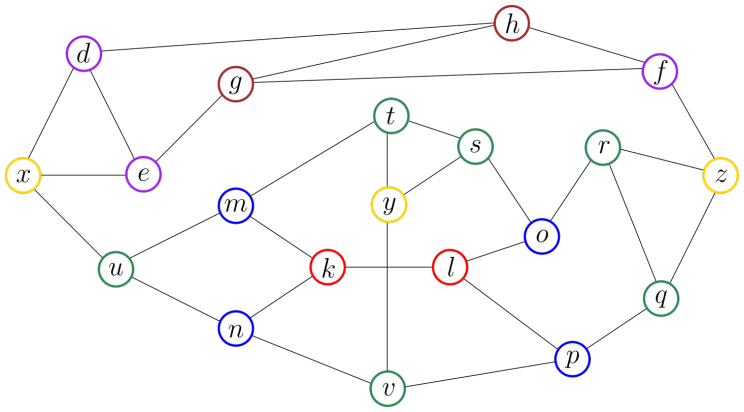


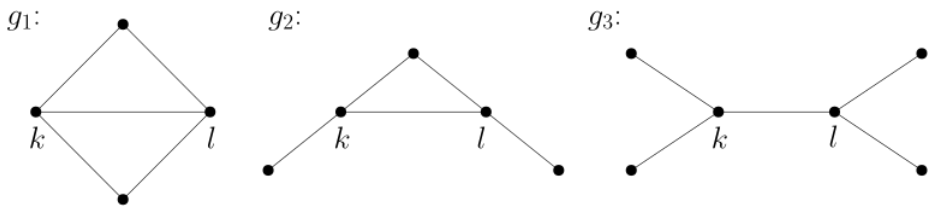
Figure 2.5: An example of a 3-regular graph. We are analyzing $F^p_{\langle kl\rangle}(\vec{\gamma}, \vec{\beta})$ – operator associated with edge $\langle kl\rangle$. The operator involves only qubit k and l (the red ones) and those qubits whose distance on the graph from k or l is less than or equal to p. In this graph, qubits whose distance from k or l is **1** are the blue ones – $\{m, n, o, p\}$, qubits whose distance is **2** are the green ones – $\{q, r, s, t, u, v\}$, **3** – the yellow ones – $\{x, y, z\}$, **4** – the violet ones – $\{d, e, f\}$, and **5** – the brown ones – $\{g, h\}$. Thus, $F^p_{\langle kl\rangle}(\vec{\gamma}, \vec{\beta})$ for $p \geq 5$ already involves all qubits.

Moreover, this result has useful consequences for problem instances with only a few subgraph types. As all edges with identical subgraph g have equal contribution $F^p_{\langle kl\rangle}(\vec{\gamma},\vec{\beta}) = F_g(\vec{\gamma},\vec{\beta})$ to the cost function $F_p(\vec{\gamma},\vec{\beta})$ (2.7) which our goal is to minimize, to calculate $F_p(\vec{\gamma},\vec{\beta})$, we can either sum over all $|E|$ edges or over all subgraph types g

$$F_p(\vec{\gamma},\vec{\beta}) = \frac{1}{2}\left(1 + \sum_{\langle k,l\rangle\in E} \left\langle \vec{\gamma},\vec{\beta}\right| Z_k Z_l \left|\vec{\gamma},\vec{\beta}\right\rangle\right) = \frac{1}{2}\left(1 + \sum_g w_g F_g(\vec{\gamma},\vec{\beta})\right),$$

where w_g is the number of occurrences of the subgraph g in the original edge sum.

Let us illustrate this on $p = 1$ QAOA applied to MaxCut on 3-regular graphs[5] where there are only these three distance-1 subgraphs:



Specifically, let us consider the instance depicted on Figure (2.5) and derive its cost function $F_1(\vec{\gamma},\vec{\beta})$. First, we assign subgraphs to the edges (by reading the graph (2.5)),

$$\begin{aligned}
g_1 &\rightarrow \emptyset,\\
g_2 &\rightarrow \Big\{\langle de\rangle, \langle dx\rangle, \langle ex\rangle, \langle fg\rangle, \langle qr\rangle, \langle qz\rangle, \langle rz\rangle, \langle st\rangle, \langle sy\rangle, \langle ty\rangle\Big\},\\
g_3 &\rightarrow \Big\{\langle dh\rangle, \langle eg\rangle, \langle fz\rangle, \langle gh\rangle, \langle hl\rangle, \langle kl\rangle, \langle km\rangle, \langle kn\rangle, \langle lo\rangle, \langle lp\rangle,\\
&\qquad \langle mt\rangle, \langle mu\rangle, \langle nu\rangle, \langle nv\rangle, \langle or\rangle, \langle os\rangle, \langle pq\rangle, \langle pv\rangle, \langle ux\rangle, \langle vy\rangle\Big\}.
\end{aligned}$$

and calculate the weights: $w_{g_1} = 0$, $w_{g_2} = 10$, and $w_{g_3} = 20$. We only need to minimize

$$F_1(\vec{\gamma},\vec{\beta}) = \frac{1}{2}\Big(1 + 10F_{g_2}(\gamma,\beta) + 20F_{g_3}(\gamma,\beta)\Big),$$

where

$$\begin{aligned}
F_{g_2} = 10\,\langle s|\, e^{i\gamma(Z_dZ_e+Z_dZ_x+Z_dZ_h+Z_eZ_x+Z_eZ_g)} e^{-i\beta(X_d+X_e)} C Z_dZ_e \cdot\\
\cdot\, e^{i\beta(X_d+X_e)} e^{-i\gamma(Z_dZ_e+Z_dZ_x+Z_dZ_h+Z_eZ_x+Z_eZ_g)} |s\rangle\,,
\end{aligned}$$

$$\begin{aligned}
F_{g_3} = 20\,\langle s|\, e^{i\gamma\left(Z_dZ_h+Z_dZ_x+Z_dZ_e+Z_hZ_f+Z_hZ_g\right)} e^{-i\beta(X_d+X_h)} Z_dZ_h e^{i\beta(X_d+X_h)}\\
e^{-i\gamma\left(Z_dZ_h+Z_dZ_x+Z_dZ_e+Z_hZ_f+Z_hZ_g\right)} |s\rangle\,.
\end{aligned}$$

[5] graphs where each vertex is incident to exactly 3 vertices

QAOA: Typical performance

In [11], Farhi et al. demonstrated that if we fix parameters $(\vec{\gamma}, \vec{\beta})$, then, for typical instances,

$$F_p(\vec{\gamma}, \vec{\beta}) = \left\langle \vec{\gamma}, \vec{\beta} \right| H_C \left| \vec{\gamma}, \vec{\beta} \right\rangle$$

concentrates. To illustrate, take a look at numerical results from 25 randomly-generated 20-vertex 3-regular graphs, each with the MaxCut value of exactly 26:

	Low		Random		High	
p	Mean	Std.	Mean	Std.	Mean	Std.
2	6.636	0.319	14.691	0.036	22.409	0.228
3	5.218	0.294	15.125	0.042	23.109	0.175
4	3.933	0.259	14.627	0.157	23.822	0.272
5	3.132	0.159	15.725	0.113	24.349	0.179
6	2.550	0.100	16.404	0.140	24.918	0.266
7	1.954	0.088	15.975	0.096	25.110	0.221

TABLE 1: Mean and standard deviation of objective function values across 25 random 20-node-3-regular graph instances each having a MaxCut value of 26. For each p there are 3 sets of fixed parameters. The first is chosen to give a low cost function value, the second is randomly chosen and the third is chosen to make the cost function high. In the low and high cases the angles are not the best possible, but are chosen to drive the objective function away from a random point in the landscape.

Figure 2.6: Results from [11]. Let us emphasize that for each experiment the parameters were fixed and based on a single random instance. Observe that standard deviations are small. It implies, that good parameters for one instance are good also for other instances, and bad values for one instance are bad for other instances, too.

This sugests the following strategy for random instances:

- Pick one instance.
- Work hard to get good angles.
- Use these angles for other instances.

QAOA's performance

Being already introduced to QAOA and some of its properties, let us look at its performance when being applied to some COPs. We will start with the bounded-occurrence

Max E3LIN2 problem (E3 → each constraint contains exactly 3 variables, LIN2 → each constraint is a linear equation mod 2). That is, given n variables ($z = z_1 z_2 \dots z_n$; $z_i = 0, 1$) and m equations of the type

$$(z_i + z_j + z_k) \mod 2 = \begin{cases} 0 \\ 1 \end{cases},$$

which string maximizes the number of satisfied equations? We consider the bounded-occurrence version of this problem; that is, each variable can be included at most in D constraints. Observe the storyline (as introduced in [25]) of the research on approximation ratios that can be achieved by QAOA and classical algorithms for bounded-occurrence Max E3LIN2:

2000 Håstad [43] gave a classical algorithm that achieves $r^\star = \left(\frac{1}{2} + \frac{\text{constant}}{D}\right)$.

2001 Trevisan [70] showed that the existence of an efficient $r^\star = \left(\frac{1}{2} + \frac{\text{constant}}{D^{1/2}}\right)$ classical approximation algorithm, for a sufficiently large constant, would imply P=NP.

2014 First version of [25] by Farhi et al. showed that QAOA_1 applied to an instance of E3LIN2 with m equations produces a string z that satisfies $\left(\frac{1}{2} + \frac{\text{constant}}{22D^{3/4}}\right) m$ equations. As far as we know, this is the first time that a quantum algorithm has been proved to achieve a better approximation ratio for a natural NP-hard optimization problem than the best known classical algorithm achieves!

2015 → A classical algorithm was found [4, 44] that efficiently finds a string satisfying $\left(\frac{1}{2} + \frac{\text{constant}}{D^{1/2}}\right) m$ equations.
→ In the second version [25], Farhi et al. improved their earlier analysis and showed that QAOA_1 produces z satisfying $\left(\frac{1}{2} + \frac{\text{constant}}{101D^{1/2}\ln D}\right) m$ equations. They also showed that in the typical case (where *typical* means that when specifying the instance of E3LIN2, each triple was said to sum to 0 or 1 with (equal) 50% probability), the quantum algorithm outputs z that satisfies $\left(\frac{1}{2} + \frac{\text{constant}}{2\sqrt{3e}D^{1/2}}\right) m$ equations.

So, if nothing else, QAOA at least inspired an improvement in classical algorithms for bounded-occurrence Max E3LIN2 problem.

Lastly, let us take a look at the most studied case: QAOA applied to MaxCut. In the original paper [26], Farhi et al. gave the approximation ratios for $p = 1$ and $p = 2$

QAOA applied to MaxCut on 3-regular graphs[6]: $r^\star_{p=1} \approx 0.6924$ and $r^\star_{p=2} \approx 0.7559$. This shows that even low-depth QAOA is better than random guessing ($r^\star = 0.5$) but far from the best known classical approximation algorithms, namely, Goemans-Williamson ($r^\star \approx 0.878$) [33], and, for graphs with maximum degree at most 3, Halperin-Livnat-Zwick ($r^\star \approx 0.9326$) [37]. In 2019, Hastings [38] introduced local tensor algorithms as a classical analogue of QAOA, which seem to be at least as promissing as QAOA for approximate optimization. He showed that a single step of such classical algorithm outperforms single-step QAOA on all triangle-free MaxCut instances, and achieves the same performance on Max E3LIN2. Later, it was pointed out [6] that $p = 11$ QAOA outperformes the best free-of-unproven-conjectures classical algorithms in case of MaxCut on large-girth regular graphs. However, a local tensor algorithm outperforms them, too [39].

2.2.3 Local classical algorithms

Local tensor algorithms are a class of optimization algorithms introduced in [38] as a classical analogue of QAOA. Simply stated, they can be described in the following procedure:

```
# each vertex v ∈ V is assigned a real variable x_v ∈ [−1, +1]
for v in V:
    x_v ∈ [−1, +1]   # initial values are algorithm-specific

Repeat some number of rounds:

    1. update each variable with a linear function of its value and the
    sum of the values on its neighbors,

    2. replace the value of the updated variable x_v on each vertex v ∈ V
    with g_a(x_v), where g_a(·) is some nonlinear function which depends on
    the round number a.

Obtain the result by replacing each variable with its sign.
```

Observe that QAOA is indeed very similar.

Besides the exciting competition between QAOA and these local tensor algorithms, there have also been proposals of how to improve QAOA [12, 24, 36]. All things considered, there still remains lots to be seen in area.

[6]Graphs where each vertex is incident to exactly three edges.

3 BosonSampling

Note: In this chapter, for the analysis to hold true, it is crucial for the photons to be indistinguishable.

3.1 Bosonic second quantization formalism

Second quantization provides a convenient language in which to formulate many-particle systems. Here, we give a brief summary of its fundamental concepts:

- A state with zero particles is called the **vacuum** state $|\emptyset\rangle$. We consider this state to be normalized: $\langle\emptyset|\emptyset\rangle = 1$.
- Let ψ_i, $i \in \{1, 2, \ldots, m \in \mathbb{N}\}$, be an orthonormal basis of a 1-particle Hilbert space. And let $|n_1, n_2, \ldots, n_m\rangle$ be an orthonormal basis of the Fock space, where n_i is the number of bosons in the i-th 1-particle state (the so-called *occupation number representation*).
- A **creation operator** $a_i^\dagger$ creates a boson in the i-th 1-particle state.

$$a^\dagger |n\rangle = |n+1\rangle .$$
$$a_3^\dagger |n_1, n_2, n_3, n_4\rangle = |n_1, n_2, n_3 + 1, n_4\rangle .$$

  $a_i^\dagger$ is the Hermitian adjoint of the annihilation operator a_i.
- An **annihilation operator** a_i lowers the number of bosons in state ψ_i by one.

$$a |n\rangle = n |n-1\rangle .$$
$$a_2 |n_1, n_2, n_3, n_4, n_5\rangle = n_2 |n_1, n_2 - 1, n_3, n_4, n_5\rangle .$$

  If there is no particle in state ψ_i, we obtain 0 as

$$a_i |\emptyset\rangle = 0.$$
- Creation and annihilation operators have to fulfill the following commutation relations:

$$[a_i, a_j^\dagger] = \delta_{ij}, \qquad [a_i, a_j] = [a_i^\dagger, a_j^\dagger] = 0.$$

- $N_i := a_i^\dagger a_i$ is the operator of the number of particles in the i-th state.

$$N_i \left|n\right\rangle = a_i^\dagger a_i \left|n\right\rangle = n \left|n\right\rangle .$$

- The Fock (basis) states $\left|n_1, n_2, \ldots, n_m\right\rangle$ can be expressed in the following way:

$$\left|n_1, n_2, \ldots, n_m\right\rangle = \frac{1}{\sqrt{n_1! n_2! \cdots n_m!}} (a_1^\dagger)^{n_1} (a_2^\dagger)^{n_2} \cdots (a_m^\dagger)^{n_m} \left|\emptyset\right\rangle .$$

Let us see for orselves that $\frac{1}{\sqrt{n_1! n_2! \cdots n_m!}}$ is indeed the correct normalizing constant. In other words, we ask:

$$\left\langle\emptyset\right| a_n^{n_m} \cdots a_2^{n_2} a_1^{n_1} (a_1^\dagger)^{n_1} (a_2^\dagger)^{n_2} \cdots (a_m^\dagger)^{n_m} \left|\emptyset\right\rangle =$$

$$= \left\langle\emptyset\right| a_1^{n_1} (a_1^\dagger)^{n_1} \left|\emptyset\right\rangle \otimes \left\langle\emptyset\right| a_2^{n_2} (a_2^\dagger)^{n_2} \left|\emptyset\right\rangle \otimes \cdots \otimes \left\langle\emptyset\right| a_m^{n_m} (a_m^\dagger)^{n_m} \left|\emptyset\right\rangle \stackrel{?}{=} n_1! n_2! \ldots n_m!$$

Let us start by drawing our attention to the first term: $\left\langle\emptyset\right| a_1^{n_1} (a_1^\dagger)^{n_1} \left|\emptyset\right\rangle$. A common step is to commute the operators through:

$$a_1 a_1^\dagger = 1 + a_1^\dagger a_1.$$

$n = 1$ example

$$\left\langle\emptyset\right| a_1 a_1^\dagger \left|\emptyset\right\rangle = \underbrace{\left\langle\emptyset|\emptyset\right\rangle}_{=1} + \underbrace{\left\langle\emptyset\right| a_1^\dagger}_{=0} \underbrace{a_1 \left|\emptyset\right\rangle}_{=0} = 1.$$

$n = 3$ example

$$\left\langle\emptyset\right| a_1^3 (a_1^\dagger)^3 \left|\emptyset\right\rangle = \left\langle\emptyset\right| a_1^2 \cdot \underbrace{a_1 a_1^\dagger}_{=1+a_1^\dagger a_1} \cdot (a_1^\dagger)^2 \left|\emptyset\right\rangle = \left\langle\emptyset\right| \underbrace{a_1^2 (a_1^\dagger)^2}_{=a_1\left(1+a_1^\dagger a_1\right)a_1^\dagger} \left|\emptyset\right\rangle + \left\langle\emptyset\right| \overbrace{a_1^2 a_1^\dagger}^{=a_1\left(1+a_1^\dagger a_1\right)} \underbrace{a_1 (a_1^\dagger)^2}_{=\left(1+a_1^\dagger a_1\right)a_1^\dagger} \left|\emptyset\right\rangle =$$

$$= 2 \underbrace{\left\langle\emptyset\right| a_1 a_1^\dagger \left|\emptyset\right\rangle}_{=1} + \underbrace{3 \left\langle\emptyset\right| a_1 a_1^\dagger a_1 a_1^\dagger \left|\emptyset\right\rangle}_{=1_{\langle\to 1\rangle} + 2(a_1^\dagger a_1)_{\langle\to 0\rangle} + ((a_1^\dagger)^2 a_1^2)_{\langle\to 0\rangle}} + \underbrace{\left\langle\emptyset\right| a_1 a_1^\dagger a_1 a_1^\dagger a_1 a_1^\dagger \left|\emptyset\right\rangle}_{=1} = 6.$$

Anytime a creation and annihilation operator meet, they create two scenarios. In the first one, this pair disappears into 1, and in the second one, they swap their positions. If all the creation and annihilation operators disappear into 1, we obtain $\left\langle\emptyset|\emptyset\right\rangle = 1$. Otherwise, we are left with $\left\langle\emptyset\right| a_1^\dagger \cdot \text{irrelevant something} \cdot a_1 \left|\emptyset\right\rangle = 0$. So, essentially, the question is: *How many possibilities of all the creation-annihilation pairings are there?*

We may imagine each one of the n_1 annihilation operators to have a slot for its pair on the right:

$$\underbrace{a_1 __ \cdot a_1 __ \cdot \ldots \cdot a_1 __}_{n_1} .$$

And there are $n_1!$ ways of how to place the n_1 creation operators into this "pairing slots". Hence,

$$\langle\emptyset|\, a_1^{n_1}(a_1^\dagger)^{n_1}\,|\emptyset\rangle = n_1!.$$

Analogically,

$$\langle\emptyset|\, a_2^{n_2}(a_2^\dagger)^{n_2}\,|\emptyset\rangle = n_2!,$$
$$\langle\emptyset|\, a_3^{n_3}(a_3^\dagger)^{n_3}\,|\emptyset\rangle = n_3!,$$
$$\text{and so on.}$$

Finally, we can observe that indeed

$$\begin{aligned}\langle\emptyset|\, a_n^{n_m}\cdots a_2^{n_2}a_1^{n_1}(a_1^\dagger)^{n_1}(a_2^\dagger)^{n_2}\cdots(a_m^\dagger)^{n_m}\,|\emptyset\rangle = \\ = \underbrace{\langle\emptyset|\, a_1^{n_1}(a_1^\dagger)^{n_1}\,|\emptyset\rangle}_{n_1!}\otimes\underbrace{\langle\emptyset|\, a_2^{n_2}(a_2^\dagger)^{n_2}\,|\emptyset\rangle}_{n_2!}\otimes\cdots\otimes\underbrace{\langle\emptyset|\, a_m^{n_m}(a_m^\dagger)^{n_m}\,|\emptyset\rangle}_{n_m!} = n_1!n_2!\ldots n_m!\,.\end{aligned} \tag{3.1}$$

3.2 BosonSampling

In 2011, Aaronson and Arkhipov [2] suggested the following task as being computationally hard to perform on any non-quantum device:

BosonSampling. *Let n be the number of photons, m the number of locations or "modes" a photon can be placed in, and $\Phi_{m,n}$ the set of all lists of nonnegative integers $S = (s_1, \ldots, s_m)$, where s_i represents the number of photons in the ith mode, such that $s_1 + \cdots + s_m = n$. Now, we can also explicitly write the hereby-used Fock basis as $\left\{|S\rangle = |s_1, \ldots, s_m\rangle\right\}_{S\in\Phi_{m,n}}$.*

- *The input is an $m \times m$ unitary scattering matrix $\hat{U}$.*
- *Choose the initial n-photon configuration $S \in \Phi_{m,n}$.*
- *Sample from the distribution over all possible output configurations $F = (f_1, \ldots, f_m) \in \Phi_{m,n}$ with the probabilities given by*

$$\Pr_{|S\rangle}(|F\rangle) = \left|\frac{\text{Permanent}(U_{S,F})}{\sqrt{s_1!\cdots s_m!}\cdot\sqrt{f_1!\cdots f_m!}}\right|^2, \tag{3.2}$$

where $U_{S,F}$ is the $n \times n$ matrix obtained from $\hat{U}$ by taking s_i copies of the ith column of $\hat{U}$ and f_j copies of the jth row, for all $i, j \in \{1, \ldots, m\}$.

While the BosonSampling-generated distribution is intrinsic to bosonic statistics (all one needs to do is repeatedly launch the photons into an m-mode interferometer $\hat{U}$ and measure the output configuration), as the permanent is #P-hard [72, 1], classically,

unless certain computational complexity classes are equivalent (which would have serious consequences for complexity theory), this distribution is (in general) difficult to sample from.

To grasp BosonSampling better, let us analyze two examples: The Hong-Ou-Mandel effect [41] and a general case with $n = m = 3$.

The Hong-Ou-Mandel is an interesting nonlinear effect for indistinguishable particles. The setup is simple: a 2×2 interferometer, with transition amplitudes given by the Hadamard matrix, and with the state $x^\dagger y^\dagger |\emptyset\rangle = |1,1\rangle$ on the input. $x^\dagger$ and $y^\dagger$ are creation operators. $x^\dagger$ creates one photon in the first mode, and $y^\dagger$ gives rise to one photon in the second mode. Thus, initially, we have exactly one photon in each of the two input modes.

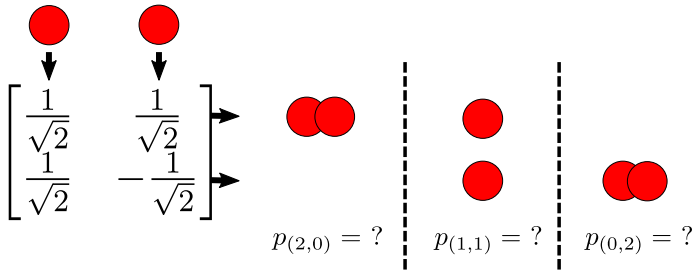


Figure 3.1: The Hong-Ou-Mandel dip – the depiction of the situation.

Hadamard, being now the scattering matrix, determines the transition:

$$x^\dagger \to \frac{1}{\sqrt{2}}(x^\dagger + y^\dagger),$$
$$y^\dagger \to \frac{1}{\sqrt{2}}(x^\dagger - y^\dagger).$$

Taking into account that bosonic creation operators commute,

$$[x^\dagger, y^\dagger] = 0 \iff x^\dagger y^\dagger = y^\dagger x^\dagger,$$

we can directly calculate the output state:

$$x^\dagger y^\dagger |\emptyset\rangle \to \left[\frac{1}{\sqrt{2}}(x^\dagger + y^\dagger)\right]\left[\frac{1}{\sqrt{2}}(x^\dagger - y^\dagger)\right]|\emptyset\rangle =$$
$$= \frac{1}{2}\left(\left(x^\dagger\right)^2 + y^\dagger x^\dagger - x^\dagger y^\dagger - \left(y^\dagger\right)^2\right)|\emptyset\rangle = \frac{1}{\sqrt{2}}\left[\frac{1}{\sqrt{2}}\left(x^\dagger\right)^2|\emptyset\rangle - \frac{1}{\sqrt{2}}\left(y^\dagger\right)^2|\emptyset\rangle\right] =$$
$$= \frac{1}{\sqrt{2}}\left(|2,0\rangle - |0,2\rangle\right).$$

Observe that the two photons will bunch together. They will never come out in distinct modes, $p_{(1,1)} = 0$.

The result we obtained is valid only for indistinguishable particles. If the particles were distinguishable, there would be four possible outcomes, each happening with

probability 1/4:

$$x^\dagger_{\text{white}} y^\dagger_{\text{black}} |\emptyset\rangle \to \left[\frac{1}{\sqrt{2}}(x^\dagger_{\text{black}} + y^\dagger_{\text{black}})\right]\left[\frac{1}{\sqrt{2}}(x^\dagger_{\text{white}} - y^\dagger_{\text{white}})\right]|\emptyset\rangle =$$
$$= \frac{1}{2}x^\dagger_{\text{black}} x^\dagger_{\text{white}} |\emptyset\rangle - \frac{1}{2}x^\dagger_{\text{black}} y^\dagger_{\text{white}} |\emptyset\rangle + \frac{1}{2}y^\dagger_{\text{black}} x^\dagger_{\text{white}} |\emptyset\rangle - \frac{1}{2}y^\dagger_{\text{black}} x^\dagger_{\text{white}} |\emptyset\rangle .$$

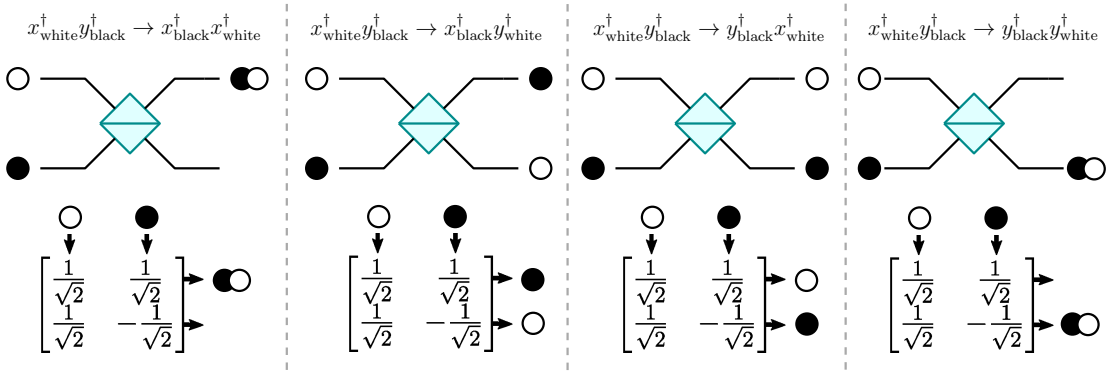


Figure 3.2: The Hong-Ou-Mandel experiment with distinguishable photons.

To emphasize the importance of indistinguishability of the photons, consider the following experiment:

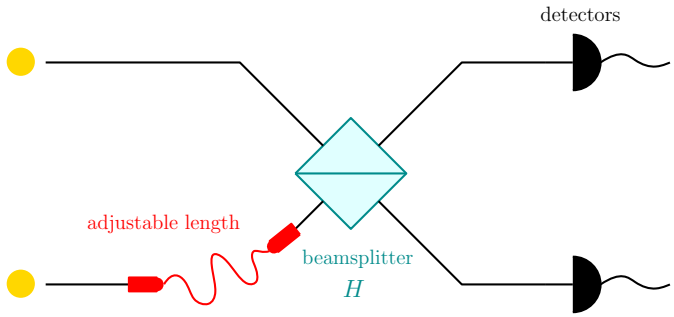


Figure 3.3: A simplified experimental setup for obtaining the Hong-Ou-Mandel interference curve. The goal is to perform the Hong-Ou-Mandel experiment: *Send two photons, each starting in a different input mode, into a beamsplitter governed by the Hadamard matrix. Measure the presence of photons in output modes.* Here's a twist though. If the two photons don't arrive into the beamsplitter simultaneously, they become distinguishable. Let us manipulate the photons' relative arrival time by adjusting the length of the second photon's incident path. How does the detectors' coincidence rate depend on the second photon's delay?

In Figure 3.4, we present the results of such, but slightly more complicated, experiment. Observe that when the photons are indistinguishable (relative distance = 0μm), there are no coincidences as the photons indeed bunch together. As the relative

incident-path difference grows, the photons gradually become more distinguishable, and the rate of coincident clicks increases, as expected.

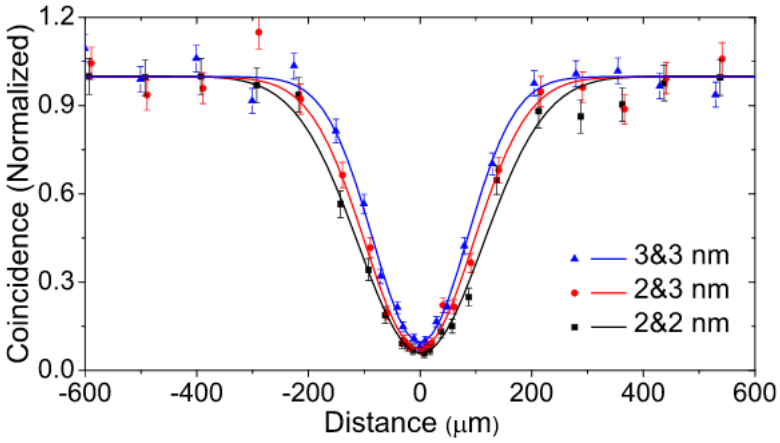


Figure 3.4: The Hong-Ou-Mandel interference curve. The probability that clicks on the two detectors that monitor output modes will coincide depends on the length difference of paths the incoming photons undergo, respectively. The visibility and the width of the Hong-Ou-Mandel dip depend on the degree of indistinguishability of the two photons of any character (e.g., different frequency, time delay between their arrival). From *Research on the Hong-Ou-Mandel interference with two independent sources*, by Si Wang et al., Sci Rep 9, 3854 (2019), https://doi.org/10.1038/s41598-019-40720-5, Figure 3. CC BY 4.0.

Let us proceed to the second example – a general case with $n = m = 3$. The total number of possibilities of how to place n photons into m slots is equivalent to the number of all ways of how to place $m-1$ slot-separators at $n+m-1$ locations (the photons occupy the remaining n locations). Thus, the number of all photon configurations can be expressed as

$$|\Phi_{m,n}| = \binom{n+m-1}{m-1} = \binom{n+m-1}{n}.$$

In our case, there are $\binom{5}{3} = 10$ configurations:

$$\begin{aligned}\Phi_{m,n} = \big\{&(3,0,0),\ (0,3,0),\ (0,0,3),\\ &(2,1,0),\ (2,0,1),\ (1,2,0),\ (0,2,1),\ (1,0,2),\ (0,1,2),\\ &(1,1,1)\big\}.\end{aligned}$$

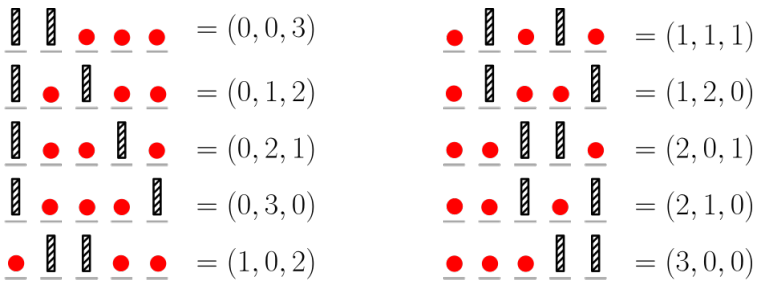


Figure 3.5: All the possibilities of placing three photons into three slots.

On the input, we are given a 3×3 unitary matrix

$$\hat{U} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}, \qquad \begin{aligned} |a|^2 + |b|^2 + |c|^2 &= 1, \\ |d|^2 + |e|^2 + |f|^2 &= 1, \\ |g|^2 + |h|^2 + |i|^2 &= 1, \end{aligned}$$

$a, b, c, d, e, f, g, h, i \in \mathbb{C}$. This scattering matrix pins down the transformation which the creation operators undergo:

$$\begin{aligned} x^\dagger &\to ax^\dagger + dy^\dagger + gz^\dagger, \\ y^\dagger &\to bx^\dagger + ey^\dagger + hz^\dagger, \\ z^\dagger &\to cx^\dagger + fy^\dagger + iz^\dagger. \end{aligned}$$

We are interested in what is the probability that three photons, being initially in the Fock state $|s_1, s_2, s_3\rangle$, will come out of the interferometer in configuration $|f_1, f_2, f_3\rangle$. As usual, the transition probability is the absolute value of the transition amplitude squared:

$$\Pr_{|s_1,s_2,s_3\rangle}(|f_1, f_2, f_3\rangle) = \left| \overbrace{\langle f_1, f_2, f_3|}^{\frac{1}{\sqrt{f_1!f_2!f_3!}} \langle\emptyset| z^{f_3} y^{f_2} x^{f_1}} \hat{U} \underbrace{|s_1, s_2, s_3\rangle}_{\frac{1}{\sqrt{s_1!s_2!s_3!}} (x^\dagger)^{s_1} (y^\dagger)^{s_2} (z^\dagger)^{s_3} |\emptyset\rangle} \right|^2.$$

In the definition of `BosonSampling`, we stated the probability is going to be

$$\Pr_{|s_1,s_2,s_3\rangle}(|f_1, f_2, f_3\rangle) = \left| \frac{\text{Permanent}\left(U_{(s_1,s_2,s_3),(f_1,f_2,f_3)}\right)}{\sqrt{s_1!s_2!s_3!} \cdot \sqrt{f_1!f_2!f_3!}} \right|^2$$

but let us verify that is the case for ourselves. To start, let us take a closer look at the final state

$$\hat{U} |s_1, s_2, s_3\rangle =$$
$$= \frac{1}{\sqrt{s_1!s_2!s_3!}} \left(ax^\dagger + dy^\dagger + gz^\dagger\right)^{s_1} \left(bx^\dagger + ey^\dagger + hz^\dagger\right)^{s_2} \left(cx^\dagger + fy^\dagger + iz^\dagger\right)^{s_3} |\emptyset\rangle .$$

Using multinomial teorem

$$(x_1 + x_2 + \ldots + x_m)^n = \sum_{k_1+k_2+\ldots+k_m=n} \frac{n!}{k_1!k_2!\ldots k_m!} \cdot \prod_{t=1}^{m} x_t^{k_t}, \tag{3.3}$$

we can express $\hat{U}\,|s_1, s_2, s_3\rangle$ in the following way:

$$\hat{U}\,|s_1, s_2, s_3\rangle =$$

$$= \frac{1}{\sqrt{s_1!s_2!s_3!}} \cdot \left(\sum_{p_1+p_2+p_3=s_1} \frac{s_1!}{p_1!p_2!p_3!} (ax^\dagger)^{p_1} (by^\dagger)^{p_2} (cz^\dagger)^{p_3} \right) \cdot$$
$$\cdot \left(\sum_{q_1+q_2+q_3=s_2} \frac{s_2!}{q_1!q_2!q_3!} (dx^\dagger)^{q_1} (ey^\dagger)^{q_2} (fz^\dagger)^{q_3} \right) \cdot$$
$$\cdot \left(\sum_{r_1+r_2+r_3=s_3} \frac{s_3!}{r_1!r_2!r_3!} (gx^\dagger)^{r_1} (hy^\dagger)^{r_2} (iz^\dagger)^{r_3} \right) |\emptyset\rangle =$$

$$= \frac{1}{\sqrt{s_1!s_2!s_3!}} \cdot \sum_{\substack{p_1+p_2+p_3=s_1 \\ q_1+q_2+q_3=s_2 \\ r_1+r_2+r_3=s_3}} \overbrace{\frac{s_1!s_2!s_3!}{p_1!p_2!p_3!q_1!q_2!q_3!r_1!r_2!r_3!}}^{=:F(p_1,p_2,p_3,q_1,q_2,q_3,r_1,r_2,r_3)} \cdot a^{p_1} b^{p_2} c^{p_3} d^{q_1} e^{q_2} f^{q_3} g^{r_1} h^{r_2} i^{r_3} \cdot$$
$$\cdot (x^\dagger)^{p_1+q_1+r_1} (y^\dagger)^{p_2+q_2+r_2} (z^\dagger)^{p_3+q_3+r_3} |\emptyset\rangle .$$

Now, we can proceed to expressing the transition amplitude:

$$\mathrm{Am}_{|s_1,s_2,s_3\rangle}(|f_1, f_2, f_3\rangle) = \langle f_1, f_2, f_3|\,\hat{U}\,|s_1, s_2, s_3\rangle =$$

$$= \sum_{\substack{p_1+p_2+p_3=s_1 \\ q_1+q_2+q_3=s_2 \\ r_1+r_2+r_3=s_3}} \frac{F(p_1, p_2, p_3, q_1, q_2, q_3, r_1, r_2, r_3)}{\sqrt{s_1!s_2!s_3!} \cdot \sqrt{f_1!f_2!f_3!}} \cdot$$
$$\cdot \langle\emptyset|\, z^{f_3} y^{f_2} x^{f_1} (x^\dagger)^{p_1+q_1+r_1} (y^\dagger)^{p_2+q_2+r_2} (z^\dagger)^{p_3+q_3+r_3} |\emptyset\rangle .$$

Unless

$$\begin{aligned} p_1 + q_1 + r_1 &= f_1, \\ p_2 + q_2 + r_2 &= f_2, \\ p_3 + q_3 + r_3 &= f_3, \end{aligned} \tag{3.4}$$

either an annihilation or creation operator will commute through and annihilate the corresponding vacuum ($a_i\,|\emptyset\rangle = \left(\langle\emptyset|\, a_i^\dagger\right)^\dagger = 0$). There simply would be no possibility of pairing all the creation operators with all the annihilation operators (to disappear into 1), hence, we would obtain 0. If all the three conditions (3.4) are satisfied,

$$\langle\emptyset|\, z^{f_3} y^{f_2} x^{f_1} (x^\dagger)^{p_1+q_1+r_1} (y^\dagger)^{p_2+q_2+r_2} (z^\dagger)^{p_3+q_3+r_3} |\emptyset\rangle =$$
$$= \langle\emptyset|\, z^{f_3} y^{f_2} x^{f_1} (x^\dagger)^{f_1} (y^\dagger)^{f_2} (z^\dagger)^{f_3} |\emptyset\rangle \overset{\text{see } 3.1}{=} f_1!f_2!f_3!.$$

Summing up, the transition amplitude is

$$\mathrm{Am}_{|s_1,s_2,s_3\rangle}(|f_1,f_2,f_3\rangle) =$$

$$= \frac{1}{\sqrt{s_1!s_2!s_3!}\cdot\sqrt{f_1!f_2!f_3!}}\cdot \overbrace{\sum_{\substack{p_1+p_2+p_3=s_1\\ q_1+q_2+q_3=s_2\\ r_1+r_2+r_3=s_3\\ p_1+q_1+r_1=f_1,\\ p_2+q_2+r_2=f_2,\\ p_3+q_3+r_3=f_3}} \frac{s_1!s_2!s_3!f_1!f_2!f_3!}{p_1!p_2!p_3!q_1!q_2!q_3!r_1!r_2!r_3!}\cdot a^{p_1}b^{p_2}c^{p_3}d^{q_1}e^{q_2}f^{q_3}g^{r_1}h^{r_2}i^{r_3}}^{=\text{Permanent}\left(U_{(s_1,s_2,s_3),(f_1,f_2,f_3)}\right)} =$$

$$= \frac{1}{\sqrt{s_1!s_2!s_3!}\cdot\sqrt{f_1!f_2!f_3!}}\cdot \text{Permanent}\begin{pmatrix} \overbrace{a & \cdots & a}^{s_1} & \overbrace{b & \cdots & b}^{s_2} & \overbrace{c & \cdots & c}^{s_3} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ a & \cdots & a & b & \cdots & b & c & \cdots & c \\ d & \cdots & d & e & \cdots & e & f & \cdots & f \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ d & \cdots & d & e & \cdots & e & f & \cdots & f \\ g & \cdots & g & h & \cdots & h & i & \cdots & i \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ g & \cdots & g & h & \cdots & h & i & \cdots & i \end{pmatrix}\begin{matrix} \\ \\ \\ \\ \\ \\ \\ \\ \end{matrix}\begin{matrix} \\ \\ f_1 \\ \\ \\ f_2 \\ \\ \\ f_3 \end{matrix}.$$

The identity,

$$\text{Permanent}\left(U_{(s_1,s_2,s_3),(f_1,f_2,f_3)}\right) = \\ = \sum_{\substack{p_1+p_2+p_3=s_1\\ q_1+q_2+q_3=s_2\\ r_1+r_2+r_3=s_3\\ p_1+q_1+r_1=f_1,\\ p_2+q_2+r_2=f_2,\\ p_3+q_3+r_3=f_3}} \frac{s_1!s_2!s_3!f_1!f_2!f_3!}{p_1!p_2!p_3!q_1!q_2!q_3!r_1!r_2!r_3!}\cdot a^{p_1}b^{p_2}c^{p_3}d^{q_1}e^{q_2}f^{q_3}g^{r_1}h^{r_2}i^{r_3},$$

deserves to be substantiated. When calculating a permanent of a matrix (eq. A.4), we must account for all the $n!$ permutations of the matrix elements such that no two elements share a column nor a row. But if $U_{(s_1,s_2,s_3),(f_1,f_2,f_3)}$ has some columns and rows repeated, several permutations will provide the same value. So, how many unique values are there? The values are characterized by:

- p_1 = the number of elements with value a (in the permutation),
- q_1 = the number of elements with value b (in the permutation),
- r_1 = the number of elements with value c (in the permutation),
- p_2 = the number of elements with value d (in the permutation),

- q_2 = the number of elements with value e (in the permutation),
- r_2 = the number of elements with value f (in the permutation),
- p_3 = the number of elements with value g (in the permutation),
- q_3 = the number of elements with value h (in the permutation),
- r_3 = the number of elements with value i (in the permutation).

And as each column and each row must provide exactly one element, it must be true that:

$$\begin{aligned} p_1 + q_1 + r_1 &= f_1 \\ p_2 + q_2 + r_2 &= f_2 \\ p_3 + q_3 + r_3 &= f_3 \\ p_1 + p_2 + p_3 &= s_1 \\ q_1 + q_2 + q_3 &= s_2 \\ r_1 + r_2 + r_3 &= s_3. \end{aligned} \tag{3.5}$$

The $n!$ permutations can provide only as many distinct values as there are $\{p_1, q_1, r_1, p_2, q_2, r_2, p_3, q_3, r_3\}$ meeting the summation conditions (3.5).

How many terms with the same value $a^{p_1}b^{p_2}c^{p_3}d^{q_1}e^{q_2}f^{q_3}g^{r_1}h^{r_2}i^{r_3}$ are there? We know how many elements from each block we need to choose:

$$\overbrace{\phantom{\begin{matrix}x\end{matrix}}}^{s_1}\ \overbrace{\phantom{\begin{matrix}x\end{matrix}}}^{s_2}\ \overbrace{\phantom{\begin{matrix}x\end{matrix}}}^{s_3}$$

$$\left(\begin{array}{c|c|c} p_1 & q_1 & r_1 \\ \hline p_2 & q_2 & r_2 \\ \hline p_3 & q_3 & r_3 \end{array}\right) \begin{matrix} \} f_1 \\ \} f_2 \\ \} f_3 \end{matrix} .$$

Let us start with the leftmost column. We first choose p_1 from the s_1 columns to have value a. There are $\binom{s_1}{p_1}$ ways to do it. Then, we choose p_2 columns from the $s_1 - p_1$ remaining columns for value d. The remaining $p_3 = s_1 - p_1 - p_2$ columns, already fully determined, are left with value g. So there are

$$\binom{s_1}{p_1} \cdot \binom{s_1 - p_1}{p_2} \cdot 1 = \frac{s_1!}{p_1!\cancel{(s_1 - p_1)!}} \cdot \frac{\cancel{(s_1 - p_1)!}}{p_2!\underbrace{(s_1 - p_1 - p_2)!}_{p_3}} = \frac{s_1!}{p_1!p_2!p_3!}$$

total ways to assign value to the leftmost s_1 columns. Analogically, there are $\frac{s_2!}{q_1!q_2!q_3!}$ ways to assign value to the middle s_2 columns, and $\frac{s_3!}{r_1!r_2!r_3!}$ ways to assign value to the rightmost s_3 columns. All in all, there are total of

$$\frac{s_1!s_2!s_3!}{p_1!p_2!p_3!q_1!q_2!q_3!r_1!r_2!r_3!}$$

ways to assign value to the columns.

Having already determined which p_1 columns have value a, which q_1 columns have value b, and which r_1 columns have value c, there is still freedom in which rows have this elements assigned. This freedom gives us $f_1!$ terms, as $p_1 + q_1 + r_1 = f_1$. Independently, we need to multiply the total number of ways of how to achieve the value $a^{p_1}b^{p_2}c^{p_3}d^{q_1}e^{q_2}f^{q_3}g^{r_1}h^{r_2}i^{r_3}$ by $f_2!$ to account for the element-row assignment for the middle $p_2 + q_2 + r_2 = f_2$ rows, and by $f_3!$ to account for the element-row assignment for the bottommost $p_3 + q_3 + r_3 = f_3$ rows. So, there are

$$\frac{s_1!s_2!s_3!f_1!f_2!f_3!}{p_1!p_2!p_3!q_1!q_2!q_3!r_1!r_2!r_3!}$$

terms with value $a^{p_1}b^{p_2}c^{p_3}d^{q_1}e^{q_2}f^{q_3}g^{r_1}h^{r_2}i^{r_3}$.

To sum up, we have indeed worked our way up to the identity

$$\begin{aligned}\text{Permanent}\left(U_{(s_1,s_2,s_3),(f_1,f_2,f_3)}\right) = \\ = \sum_{\substack{p_1+p_2+p_3=s_1\\ q_1+q_2+q_3=s_2\\ r_1+r_2+r_3=s_3\\ p_1+q_1+r_1=f_1,\\ p_2+q_2+r_2=f_2,\\ p_3+q_3+r_3=f_3}} \frac{s_1!s_2!s_3!f_1!f_2!f_3!}{p_1!p_2!p_3!q_1!q_2!q_3!r_1!r_2!r_3!} \cdot a^{p_1}b^{p_2}c^{p_3}d^{q_1}e^{q_2}f^{q_3}g^{r_1}h^{r_2}i^{r_3}.\end{aligned}$$

Hence, the transition amplitude is

$$\text{Am}_{|s_1,s_2,s_3\rangle}(|f_1,f_2,f_3\rangle) = \frac{1}{\sqrt{s_1!s_2!s_3!}\cdot\sqrt{f_1!f_2!f_3!}} \cdot \text{Permanent}\left(U_{(s_1,s_2,s_3),(f_1,f_2,f_3)}\right),$$

and the transition probability

$$\text{Pr}_{|s_1,s_2,s_3\rangle}(|f_1,f_2,f_3\rangle) = \left|\frac{\text{Permanent}\left(U_{(s_1,s_2,s_3),(f_1,f_2,f_3)}\right)}{\sqrt{s_1!s_2!s_3!}\cdot\sqrt{f_1!f_2!f_3!}}\right|^2.$$

4 Perspectives

"There's always another secret."

— Brandon Sanderson, *The Final Empire*

To conclude this thesis proposal, we introduce the subject of the author's work in her continuing doctoral studies. Listed in accordance with our priorities, we state and discuss the problems which we propose to shed a light on, and hopefully, solve in the upcoming years.

1. A new way to analyze, and understand QAOA, with performance guarantees.

To this day, only a few performance guarantees are known, and not much is understood about QAOA beyond its lowest depth versions. We intend to improve this area and find out which problems are the best fit for QAOA. So far, there were at least two observations about this:

- For QAOA_p, the measurement outcomes of any qubit depend only on the p-neighborhood of that qubit. Thus, if p is too small, QAOA does not see the whole graph [27] and its performance is limited. For large p, when QAOA can see the whole graph, there are no known indications regarding the performance limitations. So, the smaller the p required for QAOA to see the entire graph, the better. In conclusion, QAOA is best suited for geometrically highly non-local problems, with the ideal case being complete graphs.
- In [57], Ostrowski et al. showed how to reduce any combinatorial optimization problem into a *polynomial unconstrained binary optimization problem* and construct a related *hypergraph*, whose *chromatic index* gives a lower bound on the circuit depth of $e^{i\gamma H_C}$, and thus, on one round of QAOA. Therefore, the smaller the chromatic index, the more suitable the problem is. Especially, considering that errors in NISQ devices grow exponentially with the circuit depth.

Specifically, we plan to apply QAOA to various problems, focusing primarily on the suitable ones and identifying why should QAOA work, i.e., produce high approximation

ratios in polynomial time. Is it possible to determine how much each QAOA round increases the approximation ratio? What QAOA's properties are responsible for/have the most significant impact on the approximation ratio's growth? Could we formulate a new, better algorithm based on these properties? We are especially interested in for which problems (if there are such) does QAOA promise better performance guarantees than local classical algorithms introduced by Hastings [38].

2. Can GSCON for stabilizer codes be solved efficiently?

In Chapter 1, we talked about state preparation, transitions from one quantum state to another, quantum fault-tolerance, and introduced the ground state connectivity problem (GSCON). In general, GSCON is computationally very hard. To learn more about spin systems and the complexity of GSCON, we plan to study its simple variants. In particular, we wish to build up from the simplest instances to general stabilizer code Hamiltonians and answer the following question.

Stabilizer code Hamiltonians are local Hamiltonians $H_{\mathrm{SC}} = \sum_{i=1}^{m} H_i$, such that $\{H_i\}_{i=1}^{m}$ is an abelian subgroup of the n-fold Pauli group

$$\left\{ e^{i\varphi} A_1 \otimes A_2 \otimes \ldots \otimes A_n \;\middle|\; \forall j \in \{1,\ldots,n\} : A_j \in \{\mathbb{I}, X, Y, Z\} \text{ and } \varphi \in \{0, \frac{\pi}{2}, \pi, \frac{3\pi}{2}\} \right\}.$$

It turns out that the problem of finding ground states of stabilizer code Hamiltonians is in P [76]. Therefore, we can also estimate their ground state energy efficiently. For comparison, estimating ground state energies of general local Hamiltonians is QMA-complete [50]. In a similar fashion, we are interested in whether GSCON for stabilizer codes is in P, even though GSCON (in general, with m polynomial) is QCMA-complete. We will start with simple instances and aim to closely examine GSCON's connection with the topological order of the Hamiltonians.

3. Resource measure for `BosonSampling`

As the thermodynamic entropy determines whether a thermodynamical system can be transformed into another via an adiabatic process, or as the von Neumann entropy determines whether a quantum state can be transformed into another using purely local operations and classical communication, is there a physical quantity that would similarly impose a partial order on BosonSampling instances? Ideally, given two BosonSampling instances (each described by a unitary scattering matrix and an initial photon configuration), is there a quantity determining which one requires more computational steps to simulate on a classical (conventional) computer?

At least, starting with simple BosonSampling instances, can we establish a boundary where classical simulation starts to be NP-hard and where #P-hard? Is there a

class of instances (fixed by some restriction on the input photon configuration or scattering matrix) that are NP-complete to simulate? So far, it's clear that restricting the photons to enter only from a fixed constant number of input modes would cause each probability amplitude to depend on a permanent of a fixed-rank matrix, and for that reason, we could calculate any such probability amplitude in polynomial time [5]. However, the restriction doesn't affect the number of possible output photon configurations, of which there are $\binom{n+m-1}{n}$. Depending on $m(n)$, this amount might be worse than polynomial in n. So far, we know of no explicit algorithm that would generate the samples with the given probabilities in poly(n) time and what impact an exponential number of possible outcomes has; and we would like to.

Bibliography

[1] Scott Aaronson. A linear-optical proof that the permanent is #P-hard. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 467(2136):3393–3405, July 2011. DOI:10.1098/rspa.2011.0232. arXiv:1109.1674 [quant-ph].

[2] Scott Aaronson and Alex Arkhipov. The Computational Complexity of Linear Optics. *Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing*, pages 333–342, June 2011. DOI:10.1145/1993636.1993682. arXiv:1011.3245 [quant-ph].

[3] F. Arute, K. Arya, R. Babbush, D. Bacon, J.C. Bardin, R. Barends, R. Biswas, S. Boixo, F.G.S.L. Brandao, D.A. Buell, B. Burkett, Y. Chen, Z. Chen, B. Chiaro, R. Collins, W. Courtney, A. Dunsworth, E. Farhi, B. Foxen, A. Fowler, C. Gidney, M. Giustina, R. Graff, K. Guerin, S. Habegger, M.P. Harrigan, M.J. Hartmann, A. Ho, M. Hoffmann, T. Huang, T.S. Humble, S.V. Isakov, E. Jeffrey, Z. Jiang, D. Kafri, K. Kechedzhi, J. Kelly, P.V. Klimov, S. Knysh, A. Korotkov, F. Kostritsa, D. Landhuis, M. Lindmark, E. Lucero, D. Lyakh, S. Mandrà, J.R. McClean, M. McEwen, A. Megrant, X. Mi, K. Michielsen, M. Mohseni, J. Mutus, O. Naaman, M. Neeley, C. Neill, M.Y. Niu, E. Ostby, A. Petukhov, J.C. Platt, C. Quintana, E.G. Rieffel, P. Roushan, N.C. Rubin, D. Sank, K.J. Satzinger, V. Smelyanskiy, K.J. Sung, M.D. Trevithick, A. Vainsencher, B. Villalonga, T. White, Z.J. Yao, P. Yeh, A. Zalcman, H. Neven, and J.M. Martinis. Quantum supremacy using a programmable superconducting processor. *Nature*, 574:505–510, October 2019. DOI:10.1038/s41586-019-1666-5.

[4] B. Barak, A. Moitra, R. O'Donnell, P. Raghavendra, O. Regev, D. Steurer, L. Trevisan, A. Vijayaraghavan, D. Witmer, and J. Wright. Beating the random assignment on constraint satisfaction problems of bounded degree. *arXiv:1505.03424v2 [cs.CC]*, 2015.

[5] Alexander I. Barvinok. Two Algorithmic Results for the Traveling Salesman Problem. *Mathematics of Operations Research*, 21(1):65–84, 1996. URL:http://www.jstor.org/stable/3690206.

[6] Joao Basso, Edward Farhi, Kunal Marwaha, Benjamin Villalonga, and Leo Zhou. The Quantum Approximate Optimization Algorithm at High Depth for MaxCut on Large-Girth Regular Graphs and the Sherrington-Kirkpatrick Model. arXiv:2110.14206 [quant-ph], October 2022.

[7] Marcello Benedetti, Delfina Garcia-Pintos, Oscar Perdomo, Vicente Leyton-Ortega, Yunseong Nam, and Alejandro Perdomo-Ortiz. A generative modeling approach for benchmarking and training shallow quantum circuits. *npj Quantum Information*, 5(1), May 2019. DOI:10.1038/s41534-019-0157-8. arxiv:1801.07686 [quant-ph].

[8] P. Benioff. The Computer as a Physical System: A Microscopic Quantum Mechanical Hamiltonian Model of Computers as Represented by Turing Machines. *Journal of Statistical Physics 22:563–591*, May 1980. DOI:10.1007/BF01011339.

[9] Kishor Bharti, Alba Cervera-Lierta, Thi Ha Kyaw, Tobias Haug, Sumner Alperin-Lea, Abhinav Anand, Matthias Degroote, Hermanni Heimonen, Jakob S. Kottmann, Tim Menke, Wai-Keong Mok, Sukin Sim, Leong-Chuan Kwek, and Alán Aspuru-Guzik. Noisy intermediate-scale quantum algorithms. *Reviews of Modern Physics*, 94(1), Feb 2022. DOI:10.1103/revmodphys.94.015004. arXiv:2101.08448 [quant-ph].

[10] M. Born and V. A. Fock. Beweis des adiabatensatzes. *Zeitschrift für Physik A, 51(3–4):165–180*, March 1928.

[11] Fernando G.S.L Brandão, Michael Broughton, Edward Farhi, Sam Gutmann, and Hartmut Neven. For Fixed Control Parameters the Quantum Approximate Optimization Algorithm's Objective Function Value Concentrates for Typical Instances. arXiv:1812.04170 [quant-ph], December 2018.

[12] S. Bravyi, A. Kliesch, R. Koenig, and E. Tang. Obstacles to State Preparation and Variational Optimization from Symmetry Protection. *Physical Review Letters*, 125(26), 2020. DOI:10.1103/PhysRevLett.125.260505. arxiv:1910.08980 [quant-ph].

[13] Adam R. Brown and Leonard Susskind. Second law of quantum complexity. *Physical Review D*, 97(8), Apr 2018. DOI:10.1103/physrevd.97.086015. arxiv:1701.01107 [hep-th].

[14] Adam R. Brown and Leonard Susskind. Complexity geometry of a single qubit. *Physical Review D*, 100(4), Aug 2019. DOI:10.1103/physrevd.100.046020. arxiv:1903.12621 [hep-th].

[15] D. Cheung, P. Hoyer, and N. Wiebe. Improved Error Bounds for the Adiabatic Approximation. arxiv:1103.4174 [quant-ph], March 2011.

[16] I. L. Chuang, N. Gershenfeld, and M. Kubinec. Experimental Implementation of Fast Quantum Searching. *Phys. Rev. Lett. American Physical Society. 80(15):3408–3411*, April 1998. DOI:10.1103/PhysRevLett.80.3408.

[17] Arnab Das and Bikas K. Chakrabarti. Colloquium: Quantum annealing and analog quantum computation. *Reviews of Modern Physics*, 80(3):1061–1081, Sep 2008. DOI:10.1103/RevModPhys.80.1061. arXiv:0801.2193 [quant-ph].

[18] Deutsch David and Jozsa Richard. Rapid solution of problems by quantum computation. *Proc. R. Soc. Lond. A 439:553–558*, December 1992. DOI:10.1098/rspa.1992.0167.

[19] Christopher M. Dawson and Michael A. Nielsen. The Solovay-Kitaev Algorithm. *Quantum Information & Computation*, 6(1):81–95, jan January 2006. DOI:10.26421/QIC6.1-6. arxiv:quant-ph/0505030.

[20] Daoyi Dong, Chunlin Chen, Hanxiong Li, and Tzyh-Jong Tarn. Quantum Reinforcement Learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 38(5):1207–1220, 2008. DOI:10.1109/TSMCB.2008.925743. arxiv:0810.3828 [quant-ph].

[21] R. Duan. Quantum Adiabatic Theorem Revisited. arXiv:2003.03063 [quant-ph], March 2020.

[22] Vedran Dunjko, Jacob M. Taylor, and Hans J. Briegel. Quantum-Enhanced Machine Learning. *Phys. Rev. Lett.*, 117:130501, Sep 2016. DOI:10.1103/PhysRevLett.117.130501. arxiv:1610.08251 [quant-ph].

[23] Vedran Dunjko, Jacob M. Taylor, and Hans J. Briegel. Advances in quantum reinforcement learning. *2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 282–287, 2017. DOI:10.1109/SMC.2017.8122616. arxiv:1811.08676 [quant-ph].

[24] Daniel J. Egger, Jakub Mareček, and Stefan Woerner. Warm-starting quantum optimization. *Quantum*, 5:479, Jun 2021. DOI:10.22331/q-2021-06-17-479. arXiv:2009.10095 [quant-ph].

[25] E. Farhi, J. Goldstone, and S. Gutmann. A Quantum Approximate Optimization Algorithm Applied to a Bounded Occurrence Constraint Problem. arXiv:1412.6062v2 [quant-ph], June 2015.

[26] E. Farhi, J. Goldstone, and S. Gutmann. A Quantum Approximate Optimization Algorithm. arXiv:1411.4028 [quant-ph], November 2014.

[27] Edward Farhi, David Gamarnik, and Sam Gutmann. The Quantum Approximate Optimization Algorithm Needs to See the Whole Graph: A Typical Case. arXiv:2004.09002 [quant-ph], 2020.

[28] Edward Farhi and Hartmut Neven. Classification with Quantum Neural Networks on Near Term Processors. arxiv:1802.06002 [quant-ph], Feb 2018.

[29] R. Feynman. Simulating Physics with Computers. *International Journal of Theoretical Physics 21(6/7):467–488*, June 1982. DOI:10.1007/BF02650179.

[30] A.B. Finnila, M.A. Gomez, C. Sebenik, C. Stenson, and J.D. Doll. Quantum annealing: A new method for minimizing multidimensional functions. *Chemical Physics Letters*, 219(5-6):343–348, Mar 1994. DOI:10.1016/0009-2614(94)00117-0. arXiv:chem-ph/9404003.

[31] Vissarion Fisikopoulos and Luis Peñaranda. Faster geometric algorithms via dynamic determinant computation. *Computational Geometry*, 54:1–16, April 2016. DOI:10.1016/j.comgeo.2015.12.001.

[32] S. Gharibian and J. Sikora. Ground State Connectivity of Local Hamiltonians. *Automata, Languages, and Programming. ICALP 2015. Lecture Notes in Computer Science*, volume 1934, June 2015. DOI:10.1007/978-3-662-47672-7_50. arXiv:1409.3182 [quant-ph].

[33] M. X. Goemans and D. P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM (JACM), 42(6):1145*, November 1995. DOI:10.1145/227683.227684.

[34] Daniel Gottesman. Stabilizer Codes and Quantum Error Correction. arxiv:quant-ph/9705052, 1997.

[35] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '96, pages 212–219, July 1996. DOI:10.1145/237814.237866. arXiv:quant-ph/9605043.

[36] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas. From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz. *Algorithms 2019, 12(2), 34. (Special issue "Quantum Optimization Theory, Algorithms, and Applications")*, February 2019. DOI:10.3390/a12020034. arxiv:1709.03489 [quant-ph].

[37] E. Halperin, D. Livnat, and U. Zwick. MAX CUT in cubic graphs. *Journal of Algorithms, 53(2):169-185*, November 2004. DOI:10.1016/j.jalgor.2004.06.001.

[38] M. B. Hastings. Classical and Quantum Bounded Depth Approximation Algorithms. arXiv:1905.07047v2 [quant-ph], August 2019.

[39] Matthew B. Hastings. A Classical Algorithm Which Also Beats $\frac{1}{2}+\frac{2}{\pi}\frac{1}{\sqrt{d}}$ For High Girth MAX-CUT. arXiv:2111.12641 [quant-ph], November 2021.

[40] Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212, Mar 2019. DOI:10.1038/s41586-019-0980-2. arxiv:1804.11326 [quant-ph].

[41] C. K. Hong, Z. Y. Ou, and L. Mandel. Measurement of subpicosecond time intervals between two photons by interference. *Phys. Rev. Lett.*, 59:2044–2046, Nov 1987. DOI:10.1103/PhysRevLett.59.2044.

[42] Hsin-Yuan Huang, Michael Broughton, Masoud Mohseni, Ryan Babbush, Sergio Boixo, Hartmut Neven, and Jarrod R. McClean. Power of data in quantum machine learning. *Nature Communications*, 12(1), May 2021. DOI:10.1038/s41467-021-22539-9. arxiv:2011.01938 [quant-ph].

[43] J. Håstad. On bounded occurrence constraint satisfaction. *Information Processing Letters, 74(1):1-6*, 2000. DOI:10.1016/S0020-0190(00)00032-6.

[44] J. Håstad. Improved bounds for bounded occurrence constraint satisfaction. *https://www.csc.kth.se/ johanh/bounded2.pdf*, 2015.

[45] Johan Håstad. Some Optimal Inapproximability Results. *Journal of the ACM*, 48(4):798–859, July 2001. DOI: 10.1145/502090.502098.

[46] Tadashi Kadowaki and Hidetoshi Nishimori. Quantum annealing in the transverse ising model. *Physical Review E*, 58(5):5355–5363, Nov 1998. DOI:10.1103/PhysRevE.58.5355. arXiv:cond-mat/9804280 [cond-mat.stat-mech].

[47] Sami Khairy, Ruslan Shaydulin, Lukasz Cincio, Yuri Alexeev, and Prasanna Balaprakash. Learning to Optimize Variational Quantum Circuits to Solve Combinatorial Problems. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(03):2367–2375, April 2020. DOI:10.1609/aaai.v34i03.5616. arxiv:1911.11071 [cs.LG].

[48] S. Khot. On the power of unique 2-prover 1-round games. *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing, pages 767-775*, May 2002. DOI:10.1145/509907.510017.

[49] S. Khot, G. Kindler, E. Mossel, and R. O'Donnel. Optimal Inapproximability Results for MAX-CUT and Other 2-Variable CSPs? *SIAM Journal on Computing, 37(1):319-357*, 2007. DOI:10.1137/S0097539705447372.

[50] A. Yu. Kitaev, A. H. Shen, and M. N. Vyalyi. *Classical and Quantum Computation*. American Mathematical Society, USA, 2002.

[51] A.Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, Jan 2003. DOI:10.1016/s0003-4916(02)00018-0. arXiv:quant-ph/9707021.

[52] Takeru Kusumoto, Kosuke Mitarai, Keisuke Fujii, Masahiro Kitagawa, and Makoto Negoro. Experimental quantum kernel machine learning with nuclear spins in a solid. arxiv:1911.12021 [quant-ph], Nov 2019.

[53] Y. Manin. Vychislimoe i Nevychislimoe [Computable and Uncomputable] (in Russian). *Moscow, Sovetskoye Radio*, 1980.

[54] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii. Quantum circuit learning. *Phys. Rev. A*, 98:032309, Sep 2018. DOI:10.1103/PhysRevA.98.032309. arxiv:1803.00745 [quant-ph].

[55] Michael A. Nielsen. A geometric approach to quantum circuit lower bounds. arxiv:quant-ph/0502070, February 2005.

[56] Pavel Okunev and Charles R. Johnson. Necessary and Sufficient Conditions for Existence of the LU Factorization of an Arbitrary Matrix. arXiv:math/0506382 [math.NA], June 2005.

[57] James Ostrowski, Rebekah Herrman, Travis S. Humble, and George Siopsis. Lower Bounds on Circuit Depth of the Quantum Approximate Optimization Algorithm. arXiv:2004.09002 [quant-ph], 2020.

[58] J. S. Otterbach, R. Manenti, N. Alidoust, A. Bestwick, M. Block, B. Bloom, S. Caldwell, N. Didier, E. Schuyler Fried, S. Hong, P. Karalekas, C. B. Osborn, A. Papageorge, E. C. Peterson, G. Prawiroatmodjo, N. Rubin, Colm A. Ryan, D. Scarabelli, M. Scheer, E. A. Sete, P. Sivarajah, Robert S. Smith, A. Staley, N. Tezak, W. J. Zeng, A. Hudson, Blake R. Johnson, M. Reagor, M. P. da Silva, and C. Rigetti. Unsupervised Machine Learning on a Hybrid Quantum Computer. arxiv:1712.05771 [quant-ph], Dec 2017.

[59] Giuseppe Davide Paparo, Vedran Dunjko, Adi Makmal, Miguel Angel Martin-Delgado, and Hans J. Briegel. Quantum Speedup for Active Learning Agents. *Phys. Rev. X*, 4:031002, Jul 2014. DOI:10.1103/PhysRevX.4.031002. arxiv:1401.4997 [quant-ph].

[60] Edwin Pednault, John A. Gunnels, Giacomo Nannicini, Lior Horesh, and Robert Wisnieff. Leveraging secondary storage to simulate deep 54-qubit sycamore circuits. October 2019. arXiv:1910.09534 [quant-ph].

[61] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, Aug 2018. DOI:10.22331/q-2018-08-06-79. arXiv:1801.00862 [quant-ph].

[62] Giuseppe E Santoro and Erio Tosatti. Optimization using quantum mechanics: quantum annealing through adiabatic evolution. *Journal of Physics A: Mathematical and General*, 39(36):R393–R431, aug 2006. DOI:10.1088/0305-4470/39/36/r01.

[63] Maria Schuld, Kamil Brádler, Robert Israel, Daiqin Su, and Brajesh Gupt. Measuring the similarity of graphs with a gaussian boson sampler. *Phys. Rev. A*, 101:032314, Mar 2020. DOI:10.1103/PhysRevA.101.032314. arxiv:1905.12646 [quant-ph].

[64] Maria Schuld and Nathan Killoran. Quantum machine learning in feature hilbert spaces. *Phys. Rev. Lett.*, 122:040504, Feb 2019. DOI:10.1103/PhysRevLett.122.040504. arxiv:1803.07128 [quant-ph].

[65] Ruslan Shaydulin, Ilya Safro, and Jeffrey Larson. Multistart Methods for Quantum Approximate Optimization. *2019 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–8, 2019. DOI:10.1109/HPEC.2019.8916288. arxiv:1905.08768 [quant-ph].

[66] P. W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994. DOI:10.1109/SFCS.1994.365700.

[67] Peter W. Shor. Scheme for reducing decoherence in quantum computer memory. *Phys. Rev. A*, 52:R2493–R2496, Oct 1995. DOI:10.1103/PhysRevA.52.R2493.

[68] Leonard Susskind. Complexity and Gravity. *YouTube, Institute for Advanced Study, Prospects in Theoretical Physics 2018: From Qubits to Spacetime*, Jul 2018. Lecture 1: `https://www.youtube.com/watch?v=6OXdhV5BOcY`, lecture 2: `https://www.youtube.com/watch?v=EA_HOX9oMb4&list=RDLV6OXdhV5BOcY`, lecture 3: `https://www.youtube.com/watch?v=wjgkQG77pcc`.

[69] M. Suzuki. Generalized Trotter's formula and systematic approximants of exponential operators and inner derivations with applications to many-body problems. *Communications in Mathematical Physics 51:183–190*, 1976. DOI:10.1007/BF01609348.

[70] L. Trevisan. Non-approximability results for optimization problems on bounded degree instances. *Proceedings of the thirty-third annual ACM symposium on Theory of computing, pages 453–461*, July 2001. DOI:10.1145/380752.380839.

[71] Luca Trevisan, Gregory B. Sorkin, Madhu Sudan, and David P. Williamson. Gadgets, Approximation, and Linear Programming. *SIAM Journal on Computing*, 29(6):2074–2097, 2000. DOI:10.1137/S0097539797328847.

[72] L.G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2):189–201, 1979. DOI:10.1016/0304-3975(79)90044-6.

[73] Matteo M. Wauters, Emanuele Panizon, Glen B. Mbeng, and Giuseppe E. Santoro. Reinforcement-learning-assisted quantum optimization. *Physical Review Research*, 2, September 2020. DOI:10.1103/PhysRevResearch.2.033446. arxiv:2004.12323 [quant-ph].

[74] Yulin Wu, Wan-Su Bao, Sirui Cao, Fusheng Chen, Ming-Cheng Chen, Xiawei Chen, Tung-Hsun Chung, Hui Deng, Yajie Du, Daojin Fan, Ming Gong, Cheng Guo, Chu Guo, Shaojun Guo, Lianchen Han, Linyin Hong, He-Liang Huang, Yong-Heng Huo, Liping Li, Na Li, Shaowei Li, Yuan Li, Futian Liang, Chun Lin, Jin Lin, Haoran Qian, Dan Qiao, Hao Rong, Hong Su, Lihua Sun, Liangyuan Wang, Shiyu Wang, Dachao Wu, Yu Xu, Kai Yan, Weifeng Yang, Yang Yang, Yangsen Ye, Jianghan Yin, Chong Ying, Jiale Yu, Chen Zha, Cha Zhang, Haibin Zhang, Kaili Zhang, Yiming Zhang, Han Zhao, Youwei Zhao, Liang Zhou, Qingling Zhu, Chao-Yang Lu, Cheng-Zhi

Peng, Xiaobo Zhu, and Jian-Wei Pan. Strong Quantum Computational Advantage Using a Superconducting Quantum Processor. *Phys. Rev. Lett.*, 127:180501, Oct 2021. DOI:10.1103/PhysRevLett.127.180501. arXiv:2106.14734 [quant-ph].

[75] Jonathan Wurtz and Peter J. Love. Counterdiabaticity and the quantum approximate optimization algorithm. *Quantum*, 6:635, Jan 2022. Doi:10.22331/q-2022-01-27-635. arxiv:2106.15645 [quant-ph].

[76] J. Yan and D. Bacon. The k-local Pauli Commuting Hamiltonians Problem is in p. arXiv:1203.3906 [quant-ph], March 2012.

[77] Han-Sen Zhong, Yu-Hao Deng, Jian Qin, Hui Wang, Ming-Cheng Chen, Li-Chao Peng, Yi-Han Luo, Dian Wu, Si-Qiu Gong, Hao Su, Yi Hu, Peng Hu, Xiao-Yan Yang, Wei-Jun Zhang, Hao Li, Yuxuan Li, Xiao Jiang, Lin Gan, Guangwen Yang, Lixing You, Zhen Wang, Li Li, Nai-Le Liu, Jelmer J. Renema, Chao-Yang Lu, and Jian-Wei Pan. Phase-Programmable Gaussian Boson Sampling Using Stimulated Squeezed Light. *Phys. Rev. Lett.*, 127:180502, Oct 2021. DOI:10.1103/PhysRevLett.127.180502. arXiv:2106.15534 [quant-ph].

[78] Han-Sen Zhong, Hui Wang, Yu-Hao Deng, Ming-Cheng Chen, Li-Chao Peng, Yi-Han Luo, Jian Qin, Dian Wu, Xing Ding, Yi Hu, Peng Hu, Xiao-Yan Yang, Wei-Jun Zhang, Hao Li, Yuxuan Li, Xiao Jiang, Lin Gan, Guangwen Yang, Lixing You, Zhen Wang, Li Li, Nai-Le Liu, Chao-Yang Lu, and Jian-Wei Pan. Quantum computational advantage using photons. *Science*, 370(6523):1460–1463, December 2020. DOI:10.1126/science.abe8770. arXiv:2012.01625 [quant-ph].

[79] Qingling Zhu, Sirui Cao, Fusheng Chen, Ming-Cheng Chen, Xiawei Chen, Tung-Hsun Chung, Hui Deng, Yajie Du, Daojin Fan, Ming Gong, Cheng Guo, Chu Guo, Shaojun Guo, Lianchen Han, Linyin Hong, He-Liang Huang, Yong-Heng Huo, Liping Li, Na Li, Shaowei Li, Yuan Li, Futian Liang, Chun Lin, Jin Lin, Haoran Qian, Dan Qiao, Hao Rong, Hong Su, Lihua Sun, Liangyuan Wang, Shiyu Wang, Dachao Wu, Yulin Wu, Yu Xu, Kai Yan, Weifeng Yang, Yang Yang, Yangsen Ye, Jianghan Yin, Chong Ying, Jiale Yu, Chen Zha, Cha Zhang, Haibin Zhang, Kaili Zhang, Yiming Zhang, Han Zhao, Youwei Zhao, Liang Zhou, Chao-Yang Lu, Cheng-Zhi Peng, Xiaobo Zhu, and Jian-Wei Pan. Quantum computational advantage via 60-qubit 24-cycle random circuit sampling. *Science Bulletin*, 2021. DOI:110.1016/j.scib.2021.10.017. arXiv:2109.03494 [quant-ph].

A Time complexity, a brief overview

What makes some problems computationally "hard" and other problems "easy"? Complexity theory challenges us to give rigorous proofs that problems that seem to be "hard" are really "hard". It classifies problems according to their degree of "difficulty". More precisely, we ask: *What amount of computational resources (typically time or space) do variously powerful computational models need, depending on the length of the input, to solve the given problem?* Here, we will focus only on time complexity (i.e., resource = time/computational steps); and we'll restrict ourselves to basic computational models: a conventional (classical) computer and a quantum computer.

The goal of this chapter is to introduce the two most common time-classification approaches. The first one classifies *algorithms* based on the upper bound on computational steps they require to solve the problem. The second one classifies *problems* into commonly-used well-defined sets called *complexity classes*, some of the relationships among which, to this day, remain important open questions. We will introduce a subset of these classes relevant for this thesis.

A.1 Big $\mathcal{O}$ notation and time complexity

Big $\mathcal{O}$ notation is a mathematical notation that describes asymptotic upper bounds:

> *Let f and g be two increasing functions on natural numbers. Then $f \in \mathcal{O}(g)$ if and only if there exists a constant c, and some $n_0 \in \mathbb{N}$, such that for all $n \geq n_0$ it follows that: $f(n) \leq c \cdot g(n)$. Hence, g is an upper bound for f.*

It is used to assert asymptotic upper bounds on the number of steps an algorithm will take, depending on the size of the input n. We often encounter the following classes of functions:

- $\mathcal{O}(1)$ constant
- $\mathcal{O}(\log n)$ logarithmic
- $\mathcal{O}(n)$ linear
- $\mathcal{O}(n^2)$ quadratic
- $\mathcal{O}(n^c)$ polynomial
- $\mathcal{O}(c^n)$ exponential.

It is common to write "$f = \mathcal{O}(g)$" ("f is $\mathcal{O}(g)$") instead of $f \in \mathcal{O}(g)$.

The determinant

Let us start with an example of something which looks complicated to calculate, but there actually exists an $\mathcal{O}(\text{poly}(n))$ algorithm for it. The determinant of an $n \times n$ matrix $A = (a_{ij})$ is defined as

$$\det(A) = \sum_{\sigma \in S_n} \left(\text{sgn}(\sigma) \prod_{i=1}^{n} a_{i\sigma_i} \right),$$

where the sum goes through all the elements σ of the symmetric group S_n (i.e. all the $n!$ permutations of the numbers 1, 2, ..., n), and $\text{sgn}(\sigma)$ is the sign/parity of the permutation σ.

The most straightforward algorithm (= blindly follow the defining formula) is $\mathcal{O}(n \cdot n!)$ as we sum $n!$ terms, each of which needs n steps to evaluate. But there is a trick. Any square matrix A can be PLU (lower-upper with partial pivoting) decomposed [56] into a product of a permutation matrix, a lower triangular matrix and an upper triangular matrix:

$$A = PLU.$$

In the lower triangular matrix, all elements above the diagonal are zero; in the upper triangular matrix, all the elements below the diagonal are zero. Observe that evaluating $\det(L)$, as well as $\det(U)$, is easy, as in both cases, only one of the $n!$ terms is non-zero. We only need to multiply the elements on the diagonal. Another convenient fact is that $\det(P) = 1$. And as the determinant of a product is a product of determinants:

$$\det(A) = \det(PLU) = \det(P) \cdot \det(L) \cdot \det(U) = \prod_{i=1}^{n} l_{ii} \prod_{j=1}^{n} u_{jj},$$

the number of steps needed to evaluate the determinant of A is the sum of the number of steps it takes to PLU decompose A with the $\mathcal{O}(n)$ steps it takes to multiply the $2n$ matrix elements.

So, the only question left is: *How many steps does it takes to PLU decompose A?* Let us first consider only LU decomposition and let us choose Doolitle's method for this task:

```
# initialize L and U as n × n all-zero matrices
for i in {1, ..., n}:
        # determine i-th row of U
        for j in {i, ..., n}:
                u_ij = a_ij − Σ_{k=1}^{i−1} l_ik u_kj
        # determine i-th column of L
```

```
7        for j in {1,...,n}:
8            if i==j:
9                l_ii = 1
10           else:
11               l_ji = (a_ji − Σ_{k=1}^{i−1} l_jk u_ki) / u_ii
```

Doolitle's method assigns value 1 to all elements on L's main diagonal. The task is then to solve the n^2 equations given by $LU = A$.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ ? & 1 & 0 & 0 \\ ? & ? & 1 & 0 \\ ? & ? & ? & 1 \end{pmatrix} \cdot \begin{pmatrix} ? & ? & ? & ? \\ 0 & ? & ? & ? \\ 0 & 0 & ? & ? \\ 0 & 0 & 0 & ? \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix}$$

Figure A.1: $LU = A$, illustration for $n = 4$.

Observe, that the iterative manner: *determine a row of U, then a column of V, then the next row of U, the next column of V, and so on*, ensures that there is only one unknown in each equation. Namely, the equation

$$\sum_{k=1}^{n-1} l_{ik} u_{kj} = a_{ij}$$

- with $i \leq j$ will determine the element u_{ij}, as all other elements in the equation

  $$\sum_{k=1}^{i-1} l_{ik} u_{kj} + u_{ij} = a_{ij}$$

  are already known and $l_{ii} = 1$, hence,

  $$u_{ij} = a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj}; \tag{A.1}$$

- with $i > j$ will determine l_{ij}, as all other elements in the equation

  $$\sum_{k=1}^{j-1} l_{ik} u_{kj} + l_{ij} u_{jj} = a_{ij}$$

  were determined in previous steps, hence,

  $$l_{ij} = \frac{a_{ij} - \sum_{k=1}^{j-1} l_{ik} u_{kj}}{u_{jj}}. \tag{A.2}$$

Equation A.2 might pose a problem if $u_{jj} = 0$. If the matrix is invertible, such situation can be avoided by *partial pivoting*, like, for example, in this case:

$$A = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix},$$

$$A = \underbrace{\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}}_{P} \cdot \underbrace{\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}}_{L} \cdot \underbrace{\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}}_{U}.$$

The idea is to interchange the rows in each step so that $|u_{ii}|$ is as large as possible. That is, when calculating

$$u_{ii} = a_{ii} - \sum_{k=1}^{i-1} l_{ik} u_{ki}, \tag{A.3}$$

in place of a_{ii}, we try substitute all a_{ki} for $k \in \{i, \ldots, n\}$. We choose k which maximizes $|u_{ii}|$, and swap rows k and i. This approach might also help us avoid rounding errors in floating point arithmetic. For example,

$$A = \begin{pmatrix} 10^{-20} & 1 \\ 1 & 2 \end{pmatrix}$$

$$\overset{LU}{=} \begin{pmatrix} 1 & 0 \\ 10^{20} & 1 \end{pmatrix} \cdot \begin{pmatrix} 10^{-20} & 1 \\ 0 & \underbrace{2 - 10^{20}}_{\text{rounded to the nearest floating point number}} \end{pmatrix}$$

$$\overset{PLU}{=} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 \\ 10^{-20} & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 \\ 0 & -10^{-20} \end{pmatrix}.$$

However, when the matrix is singular, there must be at least one 0 on the U's main diagonal. This is because

$$\det(A) = \underbrace{\det(P)}_{=1} \cdot \underbrace{\det(L)}_{=1} \cdot \det(U) = \prod_{i=0}^{n-1} u_{ii} = 0.$$

In this case, by partial pivoting, we can still ensure that

$$\operatorname{rank}(A[\{1, \ldots, k\}, \{1, \ldots, k\}]) = \operatorname{rank}(A[\{1, \ldots, n\}, \{1, \ldots, k\}]),$$

which is a sufficient condition for A to have a PLU factorization [56]. However, for $n > 2$, the factorization will no longer be unique. For illustration, consider the following example:

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & l_{32} & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

We might choose l_{32} arbitrarily. For simplicity, we'll always choose 0.

To summarize, here is pseudocode for PLU factorization of any $n \times n$ matrix A:

```
# initialize L and U as n × n all-zero matrices
# initialize P^T as n × n identity matrix

for i in {1, ..., n}:

        # P^{-1}A = P^T A = LU
        # determine how should P^{-1} affect i-th row of A
        new_i = i
        maximal_|u_ii| = 0 # initialize to the smallest possible value
        for j in {i, ..., n}:
                |u_ii| = abs(a_ji − Σ_{k=1}^{i−1} l_ik u_kj)
                if |u_ii| > maximal_|u_ii|:
                        maximal_|u_ii| = |u_ii|
                        new_i = k
        P_i^T = # identity matrix with swapped rows i ↔ new_i
        P^T = P_i^T · P^T
        A = P_i^T · A

        # determine i-th row of U
        for j in {i, ..., n}:
                u_ij = a_ij − Σ_{k=1}^{i−1} l_ik u_kj

        # determine i-th column of L
        for j in {i, ..., n}:
                if i==j:
                        l_ii = 1
                else:
                        if u_ii == 0:
                                l_ji = 0
                        else:
                                l_ji = (a_ji − Σ_{k=1}^{i−1} l_jk u_ki) / u_ii
```

Now that we understand how the algorithm works, let us study its time complexity. For that, we'll take a closer look at two simpler examples (subparts of the algorithm). Then, we put it all together.

First, how many steps does the following program take?

```
for i in {1,...,n}:
        for j in {i,...,n}:
                u_ij = a_ij − Σ_{k=1}^{i−1} l_ik u_kj
```

When encountering for-cycles, it is best to start in the deepest nested calculations. In this case, it is evaluating u_{ij}, which takes i steps.[1] However, i is always at most n. Therefore, we can take the upper bound on the number of steps by ascribing exactly n steps to the evaluation of u_{ij} for all i, j. Taking into account both for-cycles, we need

$$\underbrace{\big(n + (n-1) + \ldots + 2 + 1\big)}_{\text{arithmetic series } \frac{n(n+1)}{2}} \cdot n = \frac{n^2(n+1)}{2}$$

steps. For n big enough, the value of n^3 is much more significant than n^2. We are interested only in asymptotic growth. For this reason, we can neglect the n^2 term. All in all, we say the algorithm is $\mathcal{O}(n^3)$.

Second, let us analyze the following code.

```
for i in {1,...,n}:
        A = P_i^T · A
```

The common approach to matrix multiplication takes $\mathcal{O}(n^3)$, as each of the n^2 matrix elements is determined by summing n terms ($A_{ij} = \sum_{k=1}^{n} P^{\mathrm{T}}_{ik} \cdot A_{kj}$). The for-cycle makes us repeat this n^3-step process n times, therefore, this example subprogram is $\mathcal{O}(n^4)$.

Now, we can look at the time complexity of the PLU factorization. It takes

$$n \cdot \Bigg(\underbrace{\sim n^2}_{\text{maximal_}|u_{ii}|} + \underbrace{\sim n^3}_{A = P_i^{\mathrm{T}} A} + \underbrace{\sim n^3}_{P^{\mathrm{T}} = P_i^{\mathrm{T}} P^{\mathrm{T}}} + \underbrace{\sim n^2}_{i\text{-th row of } U} + \underbrace{\sim n^2}_{i\text{-th column of } L} \Bigg)$$

steps. As mentioned, we are interested purely in asymptotic growth, which, in practice, means we are interested only in the term with the highest magnitude. Hence, the algorithm for the PLU factorization is $\mathcal{O}(n^4)$, which is polynomial.

To conclude, as we need to perform only $\sim n$ steps after the PLU factorization to evaluate the determinant, there is an $\mathcal{O}(n^4)$ algorithm that indeed calculates the determinant of any $n \times n$ matrix in polynomial time. The fastest algorithm for calculating the determinant is $\mathcal{O}(n^{2.373})$ [31].

[1] Although multiplication and addition demand polynomial time in the number of digits, we consider it constant, i.e., $\mathcal{O}(1)$. We assume the number of digits of the matrix elements does not depend on n.

Unstructured search

Let us also provide one simple example. Given N ***identical*** boxes, one of which contains an object we are looking for, how many steps do we need to perform (i.e., how many boxes do we need to open) to find the desired object? As there's no structure to the boxes, the best classical algorithm is also the most straightforward: *Check the boxes one by one.* In the worst case, the object will be in the very last box we open. That would mean N steps. This algorithm is, therefore, $\mathcal{O}(N)$.

A.2 Complexity classes

Many complexity classes are defined as sets of *decision problems*. So, in this brief computational-complexity overview, we first define *decision problems* and discuss whether the complexity of finding a solution is related to solving the decision-version of the problem. Next, we explain what it means to *reduce* one problem to another. Then, we provide definitions of widely used classical, as well as quantum, complexity classes. And in the end, we give an example of (many-one) reduction and introduce a #P-hard problem important for bosonic statistics called the *permanent*.

- A **decision problem** is a problem that can be posed as a yes-no question of the input values. We call such a problem *decidable* if it can be solved by an algorithm.

  Now, let us ask: *Is the time complexity of finding a solution related to solving the decision-version of the problem?* It depends on what we mean by "finding a solution".

  If we want to find the maximum value of a function $f : \{0,1\}^n \to \mathbb{N}$ which we know to be upper-bounded by some value $M \in \mathbb{N}$, then they are related. That is because we can do a binary search:

```
1  L = 0
2  R = M
3  while L < R:
4      m = ⌊(L + R)/2⌋
5      if  Is max_{x∈{0,1}^n} f(x) at least m? == yes:
6          L = m
7      else:
8          R = m
```

  which makes $\mathcal{O}(\log_2(M))$ calls to the decision-version of the problem, as we halve the interval $[L, R]$ in each step. Presuming M is not double exponential in n, or

worse, this means that finding $\max_{x\in\{0,1\}^n} f(x)$ is polynomially Turing reducible to the decision version of the problem (i.e., *Is* $\max_{x\in\{0,1\}^n} f(x)$ *at least* $\mathtt{m}$*?*). It is even easier to polynomially Turing reduce the decision problem to finding $\max_{x\in\{0,1\}^n} f(x)$. Therefore, the two problems are polynomially equivalent.

However, if by finding a solution we mean finding $x \in \{0,1\}^n$ that maximizes $f(x)$, then in general, the time complexity of the two problems is not related. For example, consider $f(x) = \max_{y\in\{0,1\}^n} g(y)$, such that $\max_{y\in\{0,1\}^n} g(y)$ is hard to compute. It is easy to find x that maximizes $f(x)$ as $f(x)$ is constant, but to decide whether there exists an x such that $f(x) > m$ is hard. Or, in a slightly different context, knowing there exists a restaurant order composed purely from appetizers worth exactly \$15.05 gives us little guidance on how to compose the order.

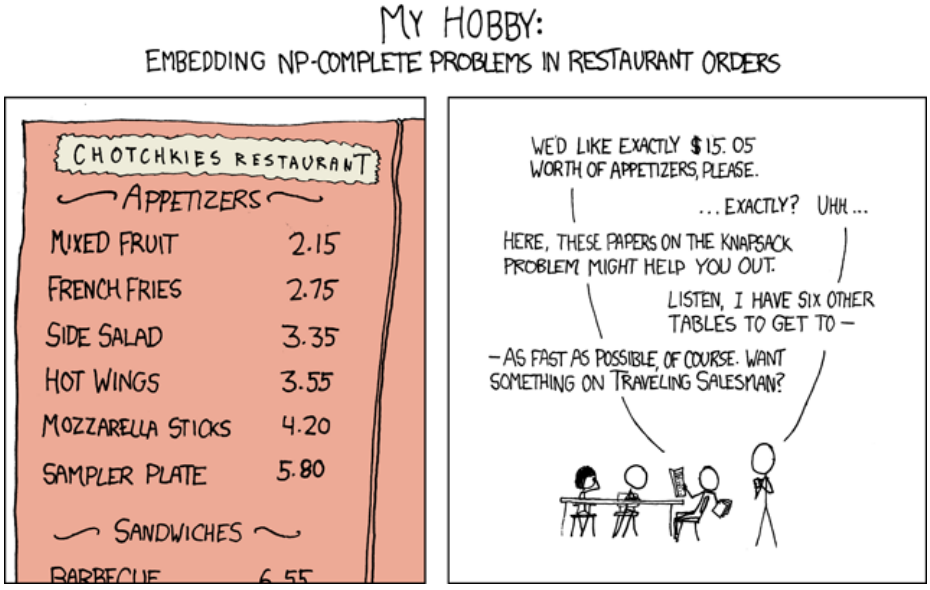


Figure A.2: Illustration of the knapsack problem from xkcd.

- A **polynomial-time Turing reduction** from a problem P to a problem R is an algorithm A that solves P by using a hypothetical subroutine S for solving R, such that if S were a polynomial-time algorithm for R, then A would be a polynomial-time algorithm for P. If P is Turing-reducible to R, we write $P \leq_T R$. If only a single call to the subroutine S for solving R is needed, and the output from S is already also the output for P,[2] such stricter reduction is called **polynomial-time many-one** (or **Karp reduction**), and we write $P \leq_m R$.

  A polynomial-time reduction from P to R proves that problem P is no more difficult than R, because whenever an efficient algorithm exists for R, one exists

[2]In other words, there exists a polynomial-time algorithm f that transforms an input x for P into an input $f(x)$ for R, such that $P(x) = R(f(x))$ for all valid inputs x.

for the P as well. If no efficient algorithm exists for P, none exists for R either.

Observe that anytime $P \leq_m R$, then also $P \leq_T R$. However, we cannot reverse this implication. For this reason, many-one reductions provide a finer time-complexity classification of problems than Turing reductions, which makes them more convenient for this purpose.

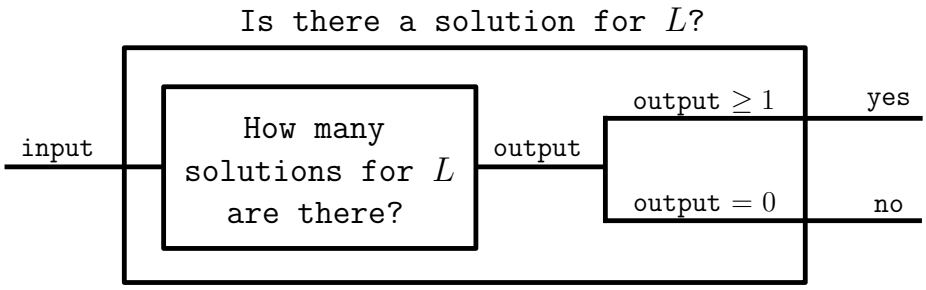


Figure A.3: A simple example of polymomial-time Turing reduction. Notice that even though only a single call to S is needed, this reduction is not many-to-one as we need to transform the output from R (ask whether it's greater than zero).

Now, let us introduce the two most famous time complexity classes: P and NP. The P versus NP problem is one of the seven Millennium Prize Problems, and thus carries a $\$1,000,000$ prize for the first correct solution. We also define the #P complexity class, as the third chapter of this thesis is devoted to a sampling problem (BosonSampling) which is #P-hard to simulate.

P	Decision problems that can be solved on a classical computer in polynomial time (polynomial number of steps).
NP	Decision problems that can be solved by a nondeterministic Turing machine in polynomial time, thence, **N**ondeterministic **P**olynomial. Or, equivalently, the set of decision problems for which the problem instances, where the answer is "yes", have proofs verifiable in polynomial time on a classical computer.
#P	Counting problems associated with the decision problems in NP. Each instance of a #P problem counts the number of solutions to some instance of the corresponding problem in NP.

Next, we present probabilistic versions of P and NP:

`BPP` The **B**ounded-error **P**robabilistic **P**olynomial time complexity class is the set of all decision problems L solvable by a classical computer A in polynomial time, such that for any valid input x one of the following applies.

1. If $L(x) = \texttt{yes}$, then the probability that A accepts x is at least $\frac{2}{3}$.
2. If $L(x) = \texttt{no}$, then the probability that A accepts x is at most $\frac{1}{3}$.

`MA` The complexity class **M**erlin-**A**rthur can be thought of as a game between all-powerful (but untrustworthy) Merlin (also known as a prover) and a standard computer Arthur (or a verifier) equipped with a random number generating device. Arthur asks Merlin a question x. Merlin replies with an answer `yes`/`no`. But Arthur doesn't trust Merlin, so Merlin also gives Arthur a simple (polynomial-time checkable) proof w that his answer is correct. A decision problem is considered to be solvable by this protocol if whenever the answer is `yes`, Merlin's responses will cause Arthur to accept at least 2/3 of the time, and if whenever the answer is `no`, Arthur will never accept more than 1/3 of the time.

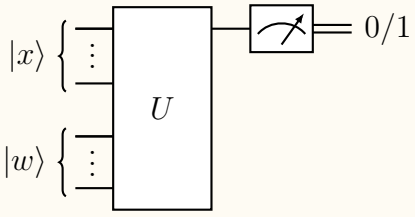
At last, we state quantum analogues of P and NP:

`BQP` The **B**ounded-error **Q**uantum **P**olynomial time complexity class is the set of all decision problems L solvable by a quantum computer A in polynomial time, such that for any valid input x one of the following applies.

1. If $L(x) = \texttt{yes}$, then the probability that A accepts x is at least $\frac{2}{3}$.
2. If $L(x) = \texttt{no}$, then the probability that A accepts x is at most $\frac{1}{3}$.

QMA The **Q**uantum **M**erlin-**A**rthur is the set of all decision problems L for which there is a polynomial time quantum verifier circuit U, such that if the answer on a (classical) input x is

- `yes`, then there exists a polynomial-size quantum proof (a quantum state) $|w\rangle \in \mathbb{C}^{\mathrm{poly}(n)}$, such that the probability that U outputs "1" on the input $|x\rangle\,|w\rangle$ is at least $\frac{2}{3}$;



- `no`, then for all quantum states (proofs) $|w\rangle$, the probability that U outputs "1" on the input $|x\rangle\,|w\rangle$ is at most $\frac{1}{3}$.

QCMA The **Q**uantum **C**lassical **M**erlin-**A**rthur is similar to `QMA`. The only difference is that the proof must be a classical string.

It is crucial not to forget to define what it means for a problem to be *hard* or *complete* for a given complexity class `C`.

- A problem L is **hard** for a class of problems `C` (L is `C`-hard) if every problem in `C` can be (polynomially) reduced to L. Thus, no problem in `C` is harder than L.
- If a problem L is both: in `C` and hard for `C`, then L is said to be **complete** for the complexity class `C` (L is `C`-complete). Thus, L is as hard as the hardest problems in `C`.

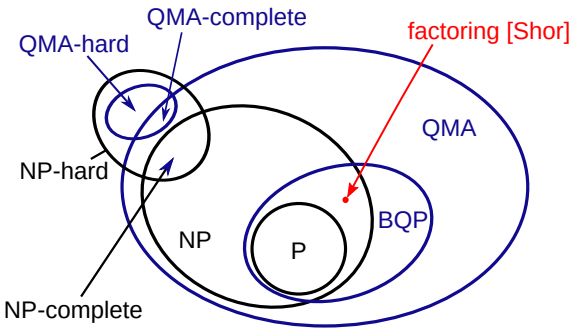


Figure A.4: A general belief of how does the Euler diagram for complexity classes P, NP, NP-complete, NP-hard, BQP, QMA, QMA-complete, and QMA-hard look like. Precise relationships are not yet known. So far, no one was even able to prove that P=NP=BQP=QMA is not the case.

The permanent

The permanent is a function very similar to the determinant. The only difference is that all the terms have positive sign. Despite this similarity, while there is a polynomial-time algorithm for the determinant, there is no known efficient algorithm for the permanent[3]. In fact, the permanent was proven to be #P-hard [72, 1]. In the remainder of this section, we define this function and demonstrate that the number of *perfect matchings* in a bipartite graph

$$G = (V - \text{the set of vertices}, E - \text{the set of edges})$$

is equal to the permanent of the *adjacency* matrix of G.

The permanent of an $n \times n$ matrix $A = (a_{i,j})$ is defined as

$$\text{Permanent}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^{n} a_{i,\sigma_i}, \tag{A.4}$$

where the sum goes through all the elements σ of the symmetric group S_n (i.e. all the $n!$ permutations of the numbers 1, 2, ..., n). Let us state also other definitions we'll need.

- A *matching* (or *independent edge set*) is any subset $M \subset E$ satisfying the condition that no two edges in M share a vertex.

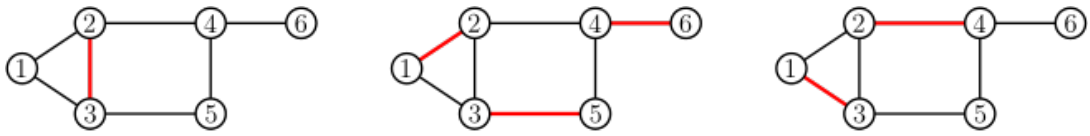


Figure A.5: Three examples of a matching in the graph $G_1 = \Big(V = \{1,2,3,4,5,6\}, E = \{\langle 1,2\rangle, \langle 1,3\rangle, \langle 2,3\rangle, \langle 2,4\rangle, \langle 3,5\rangle, \langle 4,5\rangle, \langle 4,6\rangle\}\Big)$, listed from left to right (in figure depicted in red): $M_1 = \{\langle 2,3\rangle\}$, $M_2 = \{\langle 1,2\rangle, \langle 3,5\rangle, \langle 4,6\rangle\}$, and $M_3 = \{\langle 1,3\rangle, \langle 2,4\rangle\}$.

- A *perfect matching* is a matching that matches all vertices of the graph $G = (V, E)$, i.e., every vertex in V is incident to an edge of the matching.

[3]We cannot use the same trick we used for the determinant, as for the permanent, the identity $\text{Permanent}(A \cdot B) = \text{Permanent}(A) \cdot \text{Permanent}(B)$, in general, does not hold.

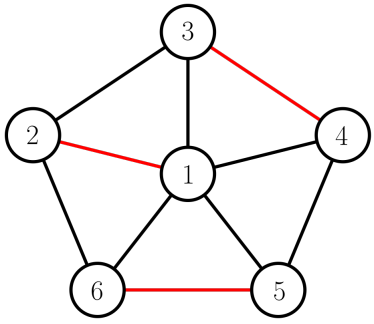


Figure A.6: $M = \{\langle 1,2\rangle, \langle 3,4\rangle, \langle 5,6\rangle\}$ is one of five perfect matchings in the graph $G_2 = \Big(V = \{1,2,3,4,5,6\}, E = \{\langle 1,2\rangle, \langle 1,3\rangle, \langle 1,4\rangle, \langle 1,5\rangle, \langle 1,6\rangle, \langle 2,3\rangle, \langle 3,4\rangle, \langle 4,5\rangle, \langle 5,6\rangle\}\Big)$.

- Given a graph $G = \Big(V = \{v_1, v_2, \ldots, v_n\}, E\Big)$, the *adjacency matrix* is a square $n \times n$ symmetric (0,1)-matrix A, such that its element

$$a_{ij} = \begin{cases} 1 \text{ if there is an edge } e \in E \text{ connecting vertex } v_i \text{ with vertex } v_j, \\ 0 \text{ otherwise.} \end{cases}$$

  For example, the adjacency matrix A_1 for graph G_1 from Figure A.5 and A_2 for graph G_2 from Figure A.6 are

$$A_1 = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}, \qquad A_2 = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

- A *bipartite graph* is a graph $G = (V, E)$ whose vertices can be divided into two disjoint and independent sets U and W (i.e., $U \cap W = \emptyset$ and $U \cup W = V$), such that every edge $e \in E$ connects a vertex in U to one in V.

- Given a bipartite graph $G = \Big((U = \{u_1, \ldots, u_k\}, W = \{w_1, \ldots, w_l\}), E\Big)$, we take the *adjacency matrix* to be a $k \times l$ (0,1)-matrix A, such that its element

$$a_{ij} = \begin{cases} 1 \text{ if there is an edge } e \in E \text{ connecting vertex } u_i \text{ with vertex } w_j, \\ 0 \text{ otherwise.} \end{cases}$$

As every edge in a bipartite graph connects some vertex in U with some vertex in W and no two edges in matching can share a vertex, there cannot exist a perfect matching unless the number of vertices in U is equal to the number of vertices in W (i.e., unless $k = l = \frac{n}{2}$). If $k = l = \frac{n}{2}$, a perfect matching exists only if it is possible to choose $\frac{n}{2}$ elements a_{ij} with value 1 from the $\frac{n}{2} \times \frac{n}{2}$ adjacency matrix A, such that

no two elements share a row or a column. This set of $\frac{n}{2}$ coordinates $\{i, j\}$ uniquely identifies the set of edges that forms perfect matching. And the permanent applied on the adjacency matrix indeed returns the number of perfect matchings, as it goes through all the $\frac{n}{2}!$ possible valid sets of coordinates and increases its value by one only if the set of coordinates is a valid perfect matching.

For illustration, we provide two examples. Consider the following 6-vertex bipartite graphs.



Red vertices form set U, and blue vertices form set W. The graph on the left has

$$\text{Permanent} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = 3! = 6$$

perfect matchings, while the graph on the right has no perfect matching. As well,

$$\text{Permanent} \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix} = 0.$$

Let us conclude with a note that we have actually provided a polynomial-time many-one reduction from problem $P =$ *How many perfect matchings does the given bipartite graph have?* to problem $R =$ *What is the permanent of the given $n \times n$ (0,1)-matrix?*. And as any $n \times n$ (0,1)-matrix A is a valid adjacency matrix of some bipartite graph G and the transformation

$$\text{(0,1)-matrix } A \longleftrightarrow \text{ bipartite graph } G$$

can be done in polynomial time in both ways, not only $P \leq_m R$, but also $R \leq_m P$, and thus, these two problems are polynomially equivalent.